



NEXUS RETAIL

AUDIT • ANALYTICS • INSIGHTS



NR - 006

SQL DATABASE DESIGN

Audit Analytics Platform

Projeto de Business Intelligence
e Auditoria de Perdas

VERSÃO 1.0

JULHO 2026



Controle do Documento

Campo	Informação
Documento	NR-006
Projeto	Audit Analytics Platform
Empresa	Nexus Retail
Autor	Alessandra Lira Oliveira
Versão	1.1
Status	Approved
Data	Julho 2026

Histórico de Versões

Versão	Data	Autor	Alteração
1.0	Julho/2026	Alessandra Lira Oliveira	Primeira versão
1.1	Agosto/2026	Alessandra Lira Oliveira	Segunda versão



Sumário

Objetivo	3
Escopo.....	3
Tecnologias Utilizadas	3
Visão Geral da Arquitetura do Banco de Dados.....	4
Convenções de Desenvolvimento	6
Padrões de Segurança e Governança do Banco de Dados	17
Implementação Física das Tabelas.....	22
Dimensão dCalendario.....	24
Dimensão dEstado	40
Dimensão dLoja	46
Dimensão dSupervisor	54
Dimensão dGerente	60
Dimensão dCategoria.....	64
Dimensão dProduto	72
Tabela Fato fPerdas	83
Objetos Auxiliares do Banco de Dados.....	97
Estratégia Geral da Indexação	111
Estratégia de Carga de Integração (ETL)	117
Controle de Transações e Tratamento de Erros	123
Monitoramento e Manutenção	129
Backup e Recuperação	134
Versionamento dos Scripts	139
Plano de Implantação	144
Testes e Critérios de Validação.....	151
Considerações Finais	157
Referências Bibliográficas	160



Objetivo

O presente documento descreve a implementação física do banco de dados da plataforma **Audit Analytics Platform**, desenvolvida para a empresa fictícia **Nexus Retail**.

O objetivo é definir a estrutura relacional responsável por armazenar os dados utilizados no processo de auditoria de perdas, garantindo integridade, consistência, escalabilidade e desempenho para suportar análises operacionais e estratégicas por meio de ferramentas de Business Intelligence,

A implementação foi projetada utilizando o Microsoft SQL Server, seguindo princípios de modelagem dimensional, normalização das dimensões de apoio e boas práticas de desenvolvimento em ambientes analíticos.

Escopo

Este documento contempla:

- Estrutura física das tabelas dimensionais e tabela fato.
- Definição de chaves primárias e estrangeiras.
- Tipos de dados utilizados.
- Restrições de integridade.
- Índice recomendados.
- Convenções de nomenclatura.
- Estratégia de carga de dados.
- Objetos auxiliares (Views e Stored Produceres).
- Estratégia de manutenção do banco de dados.

Tecnologias Utilizadas

Item	Tecnologia
Banco de Dados	Microsoft SQL Server
Linguagem	T-SQL
Ferramenta de Modelagem	dbdiagram.io
Ambiente de Desenvolvimento	SQL Server Management Studio (SSMS)
ETL	Power Query

Item	Tecnologia
Visualização	Microsoft Power BI
Controle de Versão	GitHub

Visão Geral da Arquitetura do Banco de Dados

Arquitetura Geral

A plataforma Audit Analytic Platform foi desenvolvida seguindo a arquitetura clássica de Data Warehouse, utilizando um modelo dimensional do topo **Star Schema (Modelo Estrela)**.

Essa abordagem foi escolhida por oferecer elevado desempenho em consultas analíticas, simplificar a construção de indicadores no Power BI e facilitar futuras expansões da solução.

A arquitetura está organizada em três camadas principais:

Camada de Origem (Source Layer)

Representa os sistemas operacionais responsáveis pela geração dos dados utilizados nas análises.

Entre as principais fontes previstas estão:

- Sistema ERP;
- Sistema de Auditoria;
- Inventários periódicos;
- Planilhas operacionais;
- Cadastro de produto;
- Cadastro de lojas;
- Cadastro de colaboradores.

Essas informações representam os dados brutos da operação e podem possuir formatos, padrões e níveis de qualidade.

Camada de Integração (Integration Layer)

Nesta camada ocorre todo o processo de preparação dos dados.

As principais atividades executadas são:



- Extração dos dados;
- Tratamento de inconsistências;
- Padronização de formatos;
- Limpeza de registros duplicados;
- Validações das informações;
- Aplicação das regras de negócio;
- Consolidação das bases.

Essas atividades serão implementadas por meio do Microsoft Power Query, conforme especificado no documento **NR-007 – ETL Specification**.

Cama Analítica (Analytical Layer)

Após o processamento, os dados são armazenados no banco de dados SQL Server seguindo uma modelagem dimensional composta por tabelas fato e dimensões.

Essa camada é responsável por fornecer informações consolidadas para consumo pelo Microsoft Power BI.

O modelo foi projetado para suportar análises históricas, comparações temporais, indicadores estratégicos e dashboards executivos utilizados pelos diferentes níveis de gestão da Nexus Retail.

Justificativa da Arquitetura

A escolha pela arquitetura dimensional foi baseada em critérios técnicos e de negócio.

Entre os principais benefícios obtidos destacam-se:

- Alto desempenho em consultas analíticas;
- Facilidade na construção de medidas DAX;
- Simplicidade na navegação dos dados;
- Facilidade de manutenção;
- Escalabilidade para crescimento futuro;
- Redução da complexidade das consultas SQL;
- Compatibilidade nativa com ferramentas de Business Intelligence.

Essa arquitetura também permite futuras integrações com novas tabelas fato, como vendas, inventário, auditorias e movimentações financeiras, sem necessidade de remodelagem estrutural significativa.



Convenções de Desenvolvimento

Objetivo

Este capítulo estabelece os padrões técnicos adotados durante o desenvolvimento do banco de dados da plataforma **Audit Analytics Platform**, garantindo padronização, consistência, facilidade de manutenção e escalabilidade da solução.

A definição de convenções é uma prática essencial em projetos corporativos, pois assegura que todos os objetos do banco de dados sigam critérios uniformes de nomenclatura, organização e implementação. Essa padronização reduz ambiguidades, facilita a manutenção evolutiva, melhora a legibilidade dos scripts e contribui para a governança dos dados.

Todas as estruturas neste documento seguem as convenções descritas nesta seção.

Convenção de Nomenclatura

Para facilitar a identificação dos diferentes objetos do banco de dados, foi adotado um padrão de nomenclatura baseado em prefixos.

Objeto	Prefixo	Exemplo
Tabela Dimensão	d	dProduto
Tabela Fato	f	fPerdas
View	vw	vw_PerdasPorLoja
Stored Procedure	sp	sp_CarregarPerdas
Índice	IX	IX_fPerdas_Data
Chave Primária	PK	PK_dProduto
Chave Estrangeira	FK	FK_fPerdas_dProduto
Constraint	CK	CK_ValorPerda
Default	DF	DF_StatusLoja



A utilização desses prefixos permite rapidamente a finalidade de cada objeto durante atividades de desenvolvimento, manutenção e análise de desempenho.

Convenção para Nome das Tabelas

As tabelas foram organizadas em duas categorias principais:

Tabelas Dimensionais

As tabelas dimensionais representam os atributos descritivos utilizados para contextualizar as análises.

Características:

- Possuem baixa taxa de atualização.
- Contêm informações cadastrais.
- São utilizadas para filtragem e segmentação.
- São referenciadas pelas tabelas fato.

Exemplos

- dProduto
- dLoja
- dCalendario
- dCategoria
- dFornecedor

Tabelas Fato

As tabelas fato armazenam os eventos ocorridos durante a operação.

Características:

- Grande volume de registros.
- Crescimento contínuo.
- Armazenam medidas quantitativas.
- Referenciam múltiplas dimensões.

Exemplo:

- fPerdas

Convenção para Nome das Colunas

Todas as colunas foram nomeadas utilizando o padrão PascalCase, no qual cada palavra inicia com letra maiúscula, sem utilização de espaços ou caracteres especiais.



Exemplos corretos:

- ProdutoID
- CodigoProduto
- ValorPerda
- NomeCategoria
- DataAbertura

Exemplos incorretos:

- produto_id
- produtoid
- produto id
- Produto_id

A adoção desse padrão melhora a legibilidade dos scripts SQL e mantém consistência com boas práticas de desenvolvimento.

Convenção para Chaves Primárias

Todas as tabelas possuem uma chave primária identificada pelo sufixo **ID**.

Exemplos:

- ProdutoID
- LojaID
- CategoricalID
- PerdaID

As chaves primárias possuem as seguintes características:

- Tipo inteiro (INT) para dimensões.
- Tipo BIGINT para tabelas fato de grande volume.
- Valor único.
- Não permitem valores nulos.
- Criadas utilizando a propriedade IDENTITY quando aplicável.

Essa abordagem garante identificação única de cada registro e facilita a implementação dos relacionamentos.

Convenção para Chaves Estrangeiras

As chaves estrangeiras utilizam exatamente o mesmo da chave primária da tabela referenciada.

Exemplo:



A tabela dProduto possui:

ProdutoID

A tabela fPerdas também utiliza:

ProdutoID

Essa padronização facilita a compreensão dos relacionamentos e reduz erros durante o desenvolvimento.

Convenção para Tipos de Dados

A escolha dos tipos de dados foi realizada considerando desempenho, armazenamento e compatibilidade com o SQL Server.

Tipo	Utilização
INT	Identificadores e contadores
BIGINT	Grandes volumes de registros
VARCHAR	Textos de tamanho variável
CHAR	Campos de tamanho fixo, como UF
DATE	Datas sem horário
DATETIME2	Data e hora quando necessário
DECIMAL(18,2)	Valores monetários
BIT	Campos booleanos

Sempre que possível, foram evitados tipos de dados com consumo excessivo de armazenamento.

Convenção para Valores Monetários

Todos os valores financeiros utilizam o tipo:

DECIMAL(18,2)

Essa escolha garante precisão em operações financeiras, evitando problemas de arredondamento associados ao tipo FLOAT.

Exemplo:



- ValorVenda
- ValorPerda
- ValorCusto

Convenção para Datas

A plataforma utiliza uma dimensão calendário independente (dCalendario).

Sempre que possível, as tabelas fato armazenam apenas a chave da dimensão (DataID), evitando repetição de informações e facilitando análise temporais.

A utilização da dimensão calendário permite:

- Comparação entre períodos.
- Indicadores mensais.
- Acumulados anuais.
- Análise por trimestre.
- Comparativos YoY (Year over Year).

Convenção para Índices

Os índices seguem o padrão:

IX_NomeTabela_Campo

Exemplos:

IX_fPerdas_DataID

IX_fPerdas_ProdutoID

IX_dProduto_CategorialID

A nomenclatura adotada permite identificar rapidamente a finalidade de cada índice.

Convenção para Views

As Views recebem o prefixo:

vw_

Exemplos:

- vw_PerdasPorLoja
- vw_PerdasPorCategoria
- vw_PerdasMensais



As views serão utilizadas para simplificar consultas recorrentes e disponibilizar conjuntos de dados consolidados para consumo pelo Power BI.

Convenção para Stored Procedures

As Stored Procedures recebem o prefixo:

sp_

Exemplos:

- sp_CarregarCalendario
- sp_CarregarProdutos
- sp_CarregarPerdas

Esses procedimentos serão responsáveis pelas rotinas de carga, atualização e manutenção do banco de dados.

Convenção para Comentários

Todos os scripts SQL deverão conter comentários que expliquem o objetivo das principais estruturas implementadas.

Exemplo:

```
SQL

--Criação da tabela de produtos

CREATE TABLE dProduto(

    ProdutoID INT IDENTITY(1, 1) PRIMARY KEY,

    NomeProduto VARCHAR(150) NOT NULL
```

Essa prática facilita a manutenção do código por outros desenvolvedores e contribui para a documentação técnicas da solução.

Padronização de Scripts

Os scripts SQL seguem uma estrutura padronizada composta pelas seguintes etapas:

1. Comentário de identificação.
2. Verificação de existência do objeto (quando aplicável).
3. Criação do objeto.
4. Definição das restrições.
5. Criação dos índices.
6. Comentários finais.



Essa padronização facilita o versionamento e reduz inconsistências durante implantações em diferentes ambientes.

Boas Práticas Adotadas

Durante o desenvolvimento do banco de dados foram adotadas as seguintes boas práticas:

- Utilização de modelagem dimensional para otimizar consultas analíticas.
- Separação entre tabelas fato e dimensões.
- Definição de chaves substitutas (Surrogate Keys) nas dimensões.
- Uso de integridade referencial por meio de chaves estrangeiras.
- Padronização da nomenclatura de objetos e atributos.
- Utilização de tipos de dados compatíveis com o volume esperado de informações.
- Preparação da estrutura para expansão futura da plataforma, incluindo novas tabelas fato e dimensões.
- Compatibilidade com ferramentas de Business Intelligence, especialmente o Microsoft Power BI.

Padrões de Modelagem Dimensional

Introdução

A plataforma **Audit Analytic Platform** foi projetada utilizando os princípios da **Modelagem Dimensional**, metodologia amplamente empregada em soluções de Business Intelligence (BI) e Data Warehousing.

Diretamente dos modelos transacionais (OLTP), cujo objetivo é suportar operações do dia a dia, a modelagem dimensional foi concebida para otimizar consultas analíticas, facilitar a construção de indicadores de desempenho (KPIs) e proporcionar elevada performance e, ferramentas de visualização de dados, como o Microsoft Power BI.

O modelo dimensional adotado para a Nexus Retail tem como principal objetivo consolidar informações provenientes dos processos de auditoria de perdas, permitindo análise históricas, comparativas e multidimensionais sobre produtos, lojas, regiões, categorias e períodos.



Modelo Estrela (Star Schema)

A arquitetura do banco de dados segue o padrão **Star Schema (Modelo Estrela)**, composto por uma tabela fato central e diversas tabelas dimensionais.

Nesse modelo, a tabela fato concentra os eventos quantitativos da operação, enquanto as tabelas dimensão armazenam informações descritivas utilizadas para contextualizar as análises.

As principais vantagens da adoção do Modelo Estrela são:

- Simplificação das consultas analíticas;
- Redução da complexidade dos relacionamentos;
- Alto desempenho em consultas agregadas;
- Facilidade na construção de dashboards;
- Compatibilidade nativa com ferramentas de Business Intelligence;
- Maior legibilidade para analistas e desenvolvedores.

Para o projeto Audit Analytic Platform, a tabela **fPerdas** representa o núcleo da modelagem dimensional, sendo relacionada às seguintes dimensões:

- dCalendario
- dEstado, indiretamente por meio da dLoja
- dLoja
- dSupervisor
- dGerente
- dCategoria
- dProduto

Essa estrutura possibilita análises sob diferentes perspectivas sem necessidades de remodelagem da base de dados.

Granularidade da Tabela Fato

A definição da granularidade é uma das decisões mais importantes na modelagem dimensional.

Para este projeto, foi estabelecido que cada registro da tabela **fPerda** representa **uma ocorrência individual de perda identificada durante uma auditoria**.

Cada linha corresponde a um único evento, contendo informações como:

- Data da ocorrência;
- Loja responsável;
- Produto envolvido;



- Tipo de perda identificado;
- Quantidade perdida;
- Valor financeiro da perda.

Essa granularidade permite análise extremamente detalhas e possibilita agregações em diferentes níveis hierárquicos.

Exemplos:

- Perdas por loja;
- Perdas por estado;
- Perdas por região;
- Perdas por categoria;
- Perdas por produto;
- Perdas por mês;
- Perdas por gerente;
- Perdas por supervisor.

Chaves Substitutas (Surrogate Keys)

Todas as tabelas dimensionais utilizam **Surrogate Keys**, ou seja, chaves artificiais geradas pelo próprio banco de dados.

Essas chaves são independentes dos códigos utilizados pelos sistemas operacionais da empresa e possuem como principal finalidade garantir estabilidade e consistência dos relacionamentos.

Exemplo:

Sistema Operacional	Data Warehouse
Código do Produto = A100	ProdutoID = 15
Código da Loja = VIX001	LojaID = 8

A utilização de Surrogate Keys oferece diversas vantagens:

- Independência em relação aos sistemas de origem;
- Facilidade na integração de múltiplas fontes de dados;
- Redução do tamanho das chaves estrangeiras;
- Melhoria no desempenho das consultas;
- Preservação do histórico das informações.



Chaves Naturais (Natural Keys)

Embora o banco utilize Surrogate Keys como identificadores principais, os códigos originais provenientes dos sistemas operacionais continuam sendo armazenados nas dimensões.

Exemplos:

- CodigoProduto
- COdigoLoja

Esses campos são importantes para rastreabilidade, auditorias e integração com sistemas externos.

Slowly Changing Dimensions (SCD)

As tabelas dimensionais podem sofrer alterações ao longo do tempo, como mudança de gerente de uma loja, alteração do fornecedor de um produto ou atualização do nome de uma categoria.

Para tratar essas mudanças, foram considerados os conceitos de **Slowly Changing Dimensions (SCD)**.

SCD Tipo 1

No SCD Tipo 1, a informação anterior é substituída pelo novo valor, sem manutenção do histórico.

Será utilizado para atributos que não necessitam de rastreabilidade histórica, como:

- Correção de nomes;
- Ajustes ortográficos;
- Atualização de descrições.

SCD Tipo 2

O SCD Tipo 2 preserva o histórico completo das alterações por meio da criação de um novo registro.

Embora esta funcionalidade não faça parte da primeira versão da plataforma, o modelo foi projetado para permitir sua implementação em futuras evoluções, especialmente para dimensões como:

- Loja;
- Gerente;
- Supervisor;



- Produto.

Essa decisão garante escalabilidade e prepara o banco de dados para análise histórica mais sofisticadas.

Hierarquia Dimensionais

As dimensões foram organizadas em hierarquias que permitem navegação em diferentes níveis de detalhamento.

Hierarquia Geográfica

Região → Estado → Cidade → Loja

Hierarquia Temporal

Ano → Semestre → Trimestre → Mês → Semana → Dia

Essas hierarquias serão utilizadas diretamente na construção dos dashboards do Power BI, permitindo recursos como drill-down e drill-up.

Tabela Fato

A tabela **fPerdas** concentra todas as medidas quantitativas utilizadas nas análises.

As principais métricas armazenadas são:

- Quantidade de perdas;
- Valor financeiro da perda.

A tabela fato referencia todas as dimensões necessárias para contextualização dos eventos registrados.

Escalabilidade da Modelagem

A estrutura foi projetada para permitir crescimento sem necessidade de remodelagem significativa.

Entre as possíveis evoluções previstas estão:

- Inclusão da tabela fato de vendas;
- Inclusão da tabela fato de estoque;
- Inclusão da tabela fato de auditorias;
- Inclusão da dimensão Consultor de Auditoria;
- Integração com sistemas ERP;
- Integração com plataformas de e-commerce;
- Inclusão de indicadores financeiros.



Essa estratégia assegura que a plataforma possa acompanhar o crescimento da Nexus Retail sem comprometer sua arquitetura.

Padrões de Segurança e Governança do Banco de Dados

Objetivo

Este capítulo estabelece as diretrizes de segurança, governança e administração do banco de dados da plataforma **Audit Analytics Platform**, garantindo que as informações armazenadas sejam protegidas contra acessos não autorizados, alterações indevidas, perda de dados e falhas operacionais.

Além dos mecanismos técnicos implementados pelo Microsoft SQL Server, foram definidos padrões administrativos e operacionais destinados a assegurar a integridade, confidencialidade e rastreabilidade das informações utilizadas pela Nexus Retail.

Adoção dessas práticas contribui para aumentar a confiabilidade da solução, facilitar auditorias futuras e garantir conformidade com boas práticas de governança de dados.

Princípios de Segurança

A implementação do banco de dados foi baseada nos princípios clássicos da Segurança da Informação.

Confiabilidade

As informações deverão estar acessíveis apenas aos usuários devidamente autorizados.

O acesso aos dados será controlado por meio de perfis de usuários, autenticação e permissões específicas para cada função organizacional.

Integridade

Todas as informações armazenadas deverão permanecer consistentes durante todo o seu ciclo de vida.

Para garantir a integridade dos dados serão utilizados:

- Chaves Primárias (Primary Keys);
- Chaves Estrangeiras (Foreign Keys);
- Constraints;
- Validação durante os processos ETL;
- Controle de transações.



Disponibilidade

O ambiente deverá permanecer disponível para consulta sempre que necessário.

Para isso serão adotadas políticas de :

- Backup periódico;
- Monitoramento da infraestrutura;
- Rotinas de manutenção;
- Recuperação de falhas;

Rastreabilidade

Todas as alterações realizadas no banco deverão ser identificáveis.

Sempre que possível deverão ser registrados:

- Usuário responsável;
- Data da alteração;
- Objeto alterado;
- Operação executada.

Essa prática facilita processos de auditoria e investigação de incidentes.

Classificação das Informações

Os dados armazenados no banco foram classificados de acordo com seu nível de sensibilidade.

Classificação	Descrição	Exemplo
Pública	Informações sem restrição de acesso.	Nome da categoria de produtos.
Interna	Informações utilizadas apenas por colaboradores.	Cadastro de lojas.
Confidencial	Informações estratégicas da empresa.	Valores de perdas, indicadores financeiros e desempenho das lojas.
Restrita	Informações acessíveis apenas por administradores.	Estrutura do banco, scripts administrativos e credenciais técnicas.



Essa classificação orienta a definição das permissões de acesso e das políticas de proteção de dados.

Controle de Acesso

Acesso ao banco de dados deverá seguir o princípio do **menor privilégio** (*Principle of Least Privilege*), garantindo que cada usuário possua apenas permissões estritamente necessárias para desempenhar suas atividades.

Foram definidos os seguintes perfis de acesso:

Perfil	Permissões
Administrador do Banco (DBA)	Controle total sobre o ambiente, incluindo criação de objetos, manutenção e gerenciamento de usuários.
Engenheiro de Dados	Criação e manutenção de tabelas, procedures, views e processos de carga.
Desenvolvedor BI	Leitura das tabelas e criação de views analíticas.
Analista de Auditoria	Consulta aos dados e execução de relatórios.
Diretoria Executiva	Acesso somente aos dashboards e indicadores consolidados por meio do Power BI.

Essa estrutura reduz riscos operacionais e evita alteração indevidas na base de dados.

Autenticação

A autenticação será realizada utilizando os mecanismos nativos do Microsoft SQL Server.

Dependendo do ambiente de implantação, poderão ser utilizados:

- Windows Authentication;
- SQL Serve Authentication;
- Integração com Microsoft Active Directory.

A utilização do Active Directory é recomendada por permitir gerenciamento centralizado de usuários e aplicação de políticas corporativas de segurança.



Política de Senhas

As credenciais de acesso ao banco deverão obedecer às seguintes diretrizes:

- Mínimo de 12 caracteres;
- Utilização de letras maiúsculas e minúsculas;
- Utilização de números;
- Utilização de caracteres especiais;
- Expiração periódica;
- Bloqueio automático após múltiplas tentativas inválidas.

Essas recomendações seguem práticas reconhecidas para redução do risco de acessos indevidos.

Auditoria de Alterações

Alterações estruturais realizadas no banco deverão ser registradas por meio de controle de versões e documentação.

Sempre que possível, recomenda-se registrar:

- Objeto alterado;
- Data da alteração;
- Responsável técnico;
- Justificativa;
- Versão do script.

Esse procedimento facilita o gerenciamento de mudanças e aumenta a rastreabilidade do ambiente.

Governança dos Dados

A governança dos dados estabelece responsabilidades claras sobre a administração das informações.

Papel	Responsabilidade
Data Owner	Responsável pelas regras de negócio dos dados.
Data Steward	Responsável pela qualidade e padronização das informações.
DBA	Administração técnica do banco de dados.
Desenvolvedor BI	Consumo e modelagem dos dados para análises.



A definição desses papéis contribui para a manutenção da qualidade e confiabilidade das informações ao longo do tempo.

Conformidade com a LGPD

Embora o projeto da Nexus Retail seja fictício, sua arquitetura foi concebida considerando os princípios da **Lei Geral de Proteção de Dados (Lei nº13.709/2018)**.

As seguintes diretrizes foram consideradas:

- Minimização da coleta de dados pessoais;
- Controle de acesso baseado em perfis;
- Rastreabilidade de alterações;
- Proteção das informações sensíveis;
- Armazenamento seguro dos dados.

Como o foco da plataforma é a análise de perdas operacionais, o modelo foi estruturado para evitar o armazenamento de dados pessoais desnecessários.

Boas Práticas Administrativas

Para garantir a estabilidade do ambiente, recomenda-se a adoção as seguintes práticas operacionais:

- Atualização periódica do SQL Server;
- Monitoramento de utilização de CPU, memória e armazenamento;
- Reorganização periódica dos índices;
- Atualização das estatísticas do banco;
- Validação periódica dos backups;
- Revisão dos usuários ativos;
- Documentação de todas as alterações estruturais.

Essas ações reduzem riscos de indisponibilidade e contribuem para o desempenho da plataforma.

Considerações Finais

A adoção das diretrizes apresentadas neste capítulo estabelece uma base sólida para a administração do banco de dados da plataforma Audit Analytics Platform.

Ao combinar mecanismos de segurança, governança, controle de acesso e boas práticas operacionais, a Nexus Retail assegura que seu ambiente de dados esteja preparado para suportar o crescimento da organização, preservar a qualidade das



informações e oferecer suporte confiável às análises estratégicas realizadas por meio do Microsoft Power BI.

Implementação Física das Tabelas

Introdução

Este capítulo apresenta a implementação física de todas as estruturas que compõem o banco de dados da plataforma **Audit Analytic Platform**.

Para cada tabela serão descritos:

- Objetivo da tabela;
- Finalidade de negócio associadas;
- Descrição detalhada dos atributos;
- Chaves primárias e estrangeiras;
- Restrições de integridade;
- Índices recomendados;
- Script completo de criação em T-SQL;
- Observações técnicas e recomendações de manutenção.

Essa documentação tem como objetivo servir tanto com referência para a equipe de desenvolvimento quanto como documentação oficial para futuras manutenções da plataforma.

As tabelas serão apresentadas na mesma ordem em que devem ser implementadas no banco de dados, respeitando as dependências entre chaves primárias e estrangeiras.

Ordem de Implementação

Para garantir a integridade referencial durante a criação do banco de dados, recomenda-se que as tabelas sejam criadas na seguinte sequência:

Ordem	Tabela	Tipo
2	dEstado	Dimensão
3	dCidade	Dimensão
4	dSupervisor	Dimensão
5	dGerente	Dimensão



Ordem	Tabela	Tipo
6	dDepartamento	Dimensão
7	dCategoria	Dimensão
8	dMarca	Dimensão
9	dFornecedor	Dimensão
10	dProduto	Dimensão
11	dTipoPerda	Dimensão
12	dCalendario	Dimensão
13	dLoja	Dimensão
14	fPerdas	Fato

Essa sequência garante que todas as tabelas referenciadas já existam antes da criação das respectivas chaves estrangeiras.

Padrão de Documentação das Tabelas

Todas as tabelas apresentadas neste documento seguirão o mesmo modelo de documentação, conforme a estrutura abaixo:

- Objetivo**

Descrição da finalidade da tabela no contexto da plataforma.

- Regras de Negócio**

Regras aplicáveis aos dados armazenados

- Estrutura dos Campos**

Tabela contendo:

- Nome do Campo;
- Tipo de dados;
- Obrigatório;
- Chave;
- Descrição.

- Relacionamentos**

Descrição das tabelas relacionadas e da cardinalidade.

- Script SQL**

Código completo de criação da tabela.



6. Índices Recomendados

Índices sugeridos para otimização das consultas.

7. Observações Técnicas

Comentários sobre manutenção, desempenho e futuras expansões.

Dimensão dCalendario

Resumo da Tabela

Item	Informação
Nome da Tabela	dCalendario
Tipo	Dimensão
Camada	Data Warehouse
Modelo	Star Schema
Volume Estimado	Aproximadamente 7.300 registros para um período de 20 anos
Frequência de Atualização	Anual ou conforme expansão do horizonte temporal
Origem dos Dados	Gerada internamente pelo processo de carga
Responsável	Engenharia de Dados
Utilizada em	Power BI, relatórios temporais, indicadores e análises históricas
Documento Relacionado	NR-005 – Data Model Specification

Objetivo

A dimensão **dCalendario** tem como objetivo armazenar os atributos temporais utilizados nas análises da plataforma **Audit Analytics Platform**.

Sua implementação permite que cada ocorrência registrada na tabela fato fPerdas seja associada a uma data específica, possibilitando análises por dia, semana, mês, trimestre, semestre e ano.



A dimensão calendário é essencial para a construção de comparações temporais, análises históricas, indicadores acumulados e avaliações de tendência. Ela também simplifica a elaboração de medidas no Power Bi e garante que todas as análises utilizem uma estrutura temporal única e padronizada.

No modelo aprovado, a hierarquia temporal contempla os níveis Ano, Semestre, Trimestre, Mês, Semana e Dia.

Finalidade da Dimensão

A dimensão **dCalendario** fornece o contexto temporal necessário para a análise das perdas operacionais da Nexus Retail.

Por meio dessa dimensão, será possível responder a perguntas como:

- Qual foi o valor total das perdas em determinado mês?
- Como as perdas evoluíram ao longo do ano?
- Qual trimestre apresentou maior impacto financeiro?
- Houve crescimento ou redução das perdas em relação ao período anterior?
- Quais dias da semana apresentam maior quantidade de ocorrências?
- Qual foi o desempenho do primeiro semestre em comparação com o segundo?
- Como os indicadores atuais se comparam ao mesmo período do ano anterior?

Além de organizar as datas, essa dimensão evita a criação repetitiva de colunas calculadas nas ferramentas de visualização e assegura consistência entre todos os relatórios.

Regras de Negócio

A dimensão **dCalendario** deverá atender às seguintes regras:

- Cada data deverá possuir um identificador único.
- Não será permitida a existência de datas duplicadas.
- Cada registro representará um único dia do calendário.
- O calendário deverá contemplar todo o período histórico analisado pela plataforma.



- O calendário também deverá incluir datas futuras necessárias ao planejamento dos relatórios.
- O número do mês deverá estar compreendido ente 1 e 12.
- O número do trimestre deverá estar compreendido ente 1 e 4.
- O número do semestre deverá estar compreendido ente 1 e 2.
- O número do dia da semana deverá estar compreendido entre 1 e 7.
- O nome do mês deverá corresponder ao número do mês.
- A dimensão deverá ser carregada antes da tabela fato **fPerdas**.
- Datas já utilizadas pela tabela fato não deverá ser excluídas.
- A ampliação do horizonte temporal deverá ocorrer por meio de processo controlado.

Estrutura da Tabela

Os atributos Ano, Semestre, Trimestre, Mês, Semana e Dia são definidos no NR-005. Os demais campos abaixo representam detalhamentos físicos necessários para viabilizar a implementação da dimensão no SQL Server.

Campo	Tipo	Obrigatório	Chave	Descrição
DataID	INT	Sim	PK	Identificador numérico da data no formato AAAAMMDD.
DataCompleta	DATE	Sim	UK	Data completa representada pelo registro.
Dia	TINYINT	Sim	—	Número do dia do mês.
DiaSemanaNumero	TINYINT	Sim	—	Número correspondente ao dia da semana.
DiaSemanaNome	VARCHAR(20)	Sim	—	Nome do dia da semana.
SemanaAno	TINYINT	Sim	—	Número da semana no ano.



Campo	Tipo	Obrigatório	Chave	Descrição
MesNumero	TINYINT	Sim	—	Número do mês.
MesNome	VARCHAR(20)	Sim	—	Nome completo do mês.
MesAbreviado	CHAR(3)	Sim	—	Abreviação do mês.
AnoMes	CHAR(7)	Sim	—	Identificação do período no formato AAAA-MM.
Trimestre	TINYINT	Sim	—	Número do trimestre.
Semestre	TINYINT	Sim	—	Número do semestre.
Ano	SMALLINT	Sim	—	Ano correspondente à data.
EhFimDeSemana	BIT	Sim	—	Indica se a data corresponde a sábado ou domingo.

Descrição dos Campos

DataID

Identificador único da data na dimensão.

Será utilizado o formato numérico **AAAAMMDD**.

Exemplos:

- 20260101 – 1º de janeiro de 2026.
- 20260715 – 15 de julho de 2026.
- 20261231 – 31 de dezembro de 2026.

A utilização desse formato facilita a leitura técnica, a rastreabilidade e a integração com a tabela fato.

Diferentemente das demais dimensões, o **DataID** não precisa ser gerado por IDENTITY, pois seu valor deriva diretamente da própria data.

DataCompleta



Armazena a data real correspondente ao registro.

Esse campo deverá possuir valor único, impedindo que uma mesma data seja cadastrada mais de uma vez.

Dia

Representa o número do dia dentro do mês.

Valores permitidos:

- 1 a 31.

DiaSemanaNumero

Representa numericamente o dia da semana.

Para manter a padronização da plataforma, será adotada a seguinte convecção:

Número	Dia
1	Segunda-feira
2	Terça-feira
3	Quarta-feira
4	Quinta-feira
5	Sexta-feira
6	Sábado
7	Domingo

DiaSemanaNome

Armazena o nome completo do dia da semana.

Esse campo será utilizado em filtros, gráficos e análises de concentração das perdas por dia.

SemanaAno

Representa o número da semana dentro do ano.

Será utilizado em análises semanais e comparações entre períodos operacionais.



MesNumero

Representa o número do mês.

Valores permitidos:

- 1 a 12.

O campo será utilizado para garantir a ordenação cronológica correta dos nomes dos meses no Power BI.

MesNome

Armazena o nome completo do mês.

Exemplos:

- Janeiro;
- Fevereiro;
- Março;
- Julho;
- Dezembro.

MesAbreviado

Armazena a abreviação do mês com três caracteres.

Exemplos:

- Jan;
- Fev;
- Mar;
- Jul;
- Dez.

AnoMes

Identifica o período de competência no formato AAAA-MM.

Exemplos:

- 2026-01
- 2026-07



- 2026-12

Esse atributo facilita a exibição ordenada dos períodos em gráficos e tabelas.

Trimestre

Identifica o trimestre correspondente à data.

Trimestre	Meses
1	Janeiro a março
2	Abril a junho
3	Julho a setembro
4	Outubro a dezembro

Semestre

Identifica o semestre correspondente à data.

Semestre	Meses
1	Janeiro a junho
2	Julho a dezembro

Ano

Armazena o ano da data.

Esse campo será utilizado em comparação anuais, acumulados, análises histórica e medidas Year over Year.

EhFimDeSemana

Indica se a data corresponde a um sábado ou domingo.

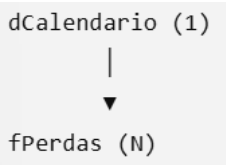
Valores:

- 0 – dia útil ou dia de semana.
- 1 – sábado ou domingo.



Relacionamentos

A dimensão **dCalendario** possui relacionamento direto do tipo **um para muitos** com a tabela fato **fPerdas**, conforme estabelecido no NR-005.



Cada data poderá estar relacionada a diversas ocorrências de perdas, enquanto cada ocorrência deverá possuir apenas uma data associada.

Cardinalidade

Origem	Destino	Cardinalidade
dCalendario	fPerdas	1:N

Script SQL

```
CREATE TABLE dCalendario
(
    DataID INT NOT NULL,
    DataCompleta DATE NOT NULL,
    Dia TINYINT NOT NULL,
    DiaSemanaNumero TINYINT NOT NULL,
    DiaSemanaNome VARCHAR(20) NOT NULL,
    SemanaAno TINYINT NOT NULL,
    MesNumero TINYINT NOT NULL,
    MesNome VARCHAR(20) NOT NULL,
    MesAbreviado CHAR(3) NOT NULL,
    AnoMes CHAR(7) NOT NULL,
    Trimestre TINYINT NOT NULL,
    Semestre TINYINT NOT NULL,
    Ano SMALLINT NOT NULL,
    EhFimDeSemana BIT NOT NULL,
    CONSTRAINT PK_dCalendario
```




```

PRIMARY KEY CLUSTERED (DataID),

CONSTRAINT UQ_dCalendario_DataCompleta
    UNIQUE (DataCompleta),

CONSTRAINT CK_dCalendario_Dia
    CHECK (Dia BETWEEN 1 AND 31),

CONSTRAINT CK_dCalendario_DiaSemana
    CHECK (DiaSemanaNumero BETWEEN 1 AND 7),

CONSTRAINT CK_dCalendario_Mes
    CHECK (MesNumero BETWEEN 1 AND 12),

CONSTRAINT CK_dCalendario_Trimestre
    CHECK (Trimestre BETWEEN 1 AND 4),


CONSTRAINT CK_dCalendario_Semestre
    CHECK (Semestre BETWEEN 1 AND 2),

CONSTRAINT CK_dCalendario_Semana
    CHECK (SemanaAno BETWEEN 1 AND 53)

);

```

Justificativa Técnica

A utilização de uma dimensão calendário independente oferece maior controle sobre as análises temporais e elimina a necessidade de extrair repetidamente ano, mês, trimestre ou dia diretamente da tabela fato.

O campo **DataID** utiliza o padrão **AAAAMMDD**, permitindo identificação simples e estável da data.

A coluna **DataCompleta** utiliza o tipo **DATE**, pois não há necessidade de armazenar horário na granularidade atual.

Os atributos numéricos foram definidos com tipos menores, como **TINYINT** e **SMALLINT**, reduzindo o consumo de armazenamento sem comprometer a capacidade dos campos.

A presença de atributos textuais e numéricos para mês e dia da semana garante facilidade de uso nos relatórios e ordenação cronológica correta.

Índice Recomendados

A chave primária já cria um índice clustered sobre **DataID**. Além dela, recomenda-se:

```
CREATE UNIQUE INDEX IX_dCalendario_DataCompleta
```



ON dCalendario (DataCompleta);

CREATE INDEX IX_dCalendario_AnoMes

ON dCalendario (Ano, MesNumero);

CREATE INDEX IX_dCalendario_AnoTrimestre

ON dCalendario (Ano, Trimestre);

Benefícios

- Otimização de consultas por período;
- Melhoria em filtros anuais e mensais;
- Aceleração de agrupamentos por trimestre;
- Facilidade de integração com o Power BI;
- Garantia contra duplicidade de datas.

Exemplo de Registros

DataID	DataCompleta	Dia	Dia da Semana	Semana	Mês	Trimestre	Semestre	Ano	Fim de Semana
20260101	01/01/2026	1	Quinta- feira	1	Janeiro	1	1	2026	0
20260102	02/01/2026	2	Sexta-feira	1	Janeiro	1	1	2026	0
20260103	03/01/2026	3	Sábado	1	Janeiro	1	1	2026	1
20260104	04/01/2026	4	Domingo	1	Janeiro	1	1	2026	1
20260105	05/01/2026	5	Segunda- feira	2	Janeiro	1	1	2026	0

Procedure para Carga do Calendário

A procedure abaixo permite gerar os registros da dimensão para um intervalo informado.

CREATE OR ALTER PROCEDURE sp_CarregarCalendario

@DataInicial DATE,

@DataFinal DATE

AS



```

BEGIN

SET NOCOUNT ON;

SET DATEFIRST 1;


IF @DataInicial IS NULL OR @DataFinal IS NULL

BEGIN

    THROW 50001,

        'As datas inicial e final são obrigatórias.',

        1;

END;


IF @DataInicial > @DataFinal

BEGIN

    THROW 50002,

        'A data inicial não pode ser superior à data final.',

        1;

END;


DECLARE @DataAtual DATE = @DataInicial;


WHILE @DataAtual <= @DataFinal

BEGIN

    IF NOT EXISTS

    (

        SELECT 1

        FROM dCalendario

        WHERE DataCompleta = @DataAtual

    )

    BEGIN

        INSERT INTO dCalendario

        (

            DataID,

            DataCompleta,

            Dia,

            DiaSemanaNumero,

```



```

DiaSemanaNome,

SemanaAno,

MesNumero,

MesNome,

MesAbreviado,

AnoMes,

Trimestre,

Semestre,

Ano,

EhFimDeSemana

)

VALUES

(

    CONVERT(INT, CONVERT(CHAR(8), @DataAtual, 112)),

    @DataAtual,

    DAY(@DataAtual),

    DATEPART(WEEKDAY, @DataAtual),

    CASE DATEPART(WEEKDAY, @DataAtual)

        WHEN 1 THEN 'Segunda-feira'

        WHEN 2 THEN 'Terça-feira'

        WHEN 3 THEN 'Quarta-feira'

        WHEN 4 THEN 'Quinta-feira'

        WHEN 5 THEN 'Sexta-feira'

        WHEN 6 THEN 'Sábado'

        WHEN 7 THEN 'Domingo'

    END,

    DATEPART(ISO_WEEK, @DataAtual),

    MONTH(@DataAtual),

    CASE MONTH(@DataAtual)

        WHEN 1 THEN 'Janeiro'

        WHEN 2 THEN 'Fevereiro'

        WHEN 3 THEN 'Março'

        WHEN 4 THEN 'Abril'

        WHEN 5 THEN 'Maio'

        WHEN 6 THEN 'Junho'

```



```

        WHEN 7 THEN 'Julho'

        WHEN 8 THEN 'Agosto'

        WHEN 9 THEN 'Setembro'

        WHEN 10 THEN 'Outubro'

        WHEN 11 THEN 'Novembro'

        WHEN 12 THEN 'Dezembro'

    END,

    CASE MONTH(@DataAtual)

        WHEN 1 THEN 'Jan'

        WHEN 2 THEN 'Fev'

        WHEN 3 THEN 'Mar'

        WHEN 4 THEN 'Abr'

        WHEN 5 THEN 'Mai'

        WHEN 6 THEN 'Jun'

        WHEN 7 THEN 'Jul'

        WHEN 8 THEN 'Ago'

        WHEN 9 THEN 'Set'

        WHEN 10 THEN 'Out'

        WHEN 11 THEN 'Nov'

        WHEN 12 THEN 'Dez'

    END,

    CONCAT(

        YEAR(@DataAtual),

        '-',

        RIGHT('0' + CAST(MONTH(@DataAtual) AS VARCHAR(2)), 2)

    ),

    DATEPART(QUARTER, @DataAtual),

    CASE

        WHEN MONTH(@DataAtual) <= 6 THEN 1

        ELSE 2

    END,

    YEAR(@DataAtual),

    CASE

        WHEN DATEPART(WEEKDAY, @DataAtual) IN (6, 7)

        THEN 1

```



```

ELSE 0

END

);

END;

SET @DataAtual = DATEADD(DAY, 1, @DataAtual);

END;

END;

```

Exemplo de execução

```

EXEC sp_CarregarCalendario

@DataInicial = '2020-01-01',

@DataFinal = '2035-12-31';

```

Consultas SQL de Exemplo

Consultar o calendário de determinado ano

```

SELECT

    DataID,

    DataCompleta,

    DiaSemanaNome,

    MesNome,

    Trimestre,

    Semestre,

    Ano

FROM dCalendario

WHERE Ano = 2026

ORDER BY DataCompleta;

```

Quantidade de dias por mês

```

SELECT

    Ano,

    MesNumero,

    MesNome,

    COUNT(*) AS QuantidadeDias

FROM dCalendario

GROUP BY

    Ano,

```



```
MesNumero,  
  
MesNome  
  
ORDER BY  
  
Ano,  
  
MesNumero;
```

Datas correspondentes a finais de semana

```
SELECT  
  
DataCompleta,  
  
DiaSemanaNome  
  
FROM dCalendario  
  
WHERE EhFimDeSemana = 1  
  
ORDER BY DataCompleta;
```

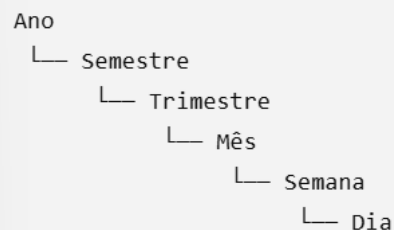
Objetivo

A dimensão **dCalendario** deverá ser marcada como tabela de datas no Power BI, utilizando a coluna **DataCompleta**.

Ela será utilizada em:

- Segmentadores por ano, mês e período;
- Gráficos de evolução mensal;
- Comparação entre períodos;
- Acumulados mensais e anuais;
- Indicadores Yar to Date;
- Análise por trimestre e semestre;
- Drill-down temporal;
- Cálculos com funções de inteligência de tempo em DAX.

A hierarquia sugerida será:



Impacto Analítico

A dimensão calendário possibilita analisar as perdas em diferentes níveis temporais e identificar tendências que não seriam percebidas em uma análise exclusivamente consolidada.

Entre os principais impactos estão:

- Identificação de meses com maior incidência de perdas;
- Comparação entre semestres;
- Avaliação de sazonalidade;
- Identificação de crescimento ou redução dos prejuízos;
- Comparação com o mesmo período do ano anterior;
- Acompanhamento da efetividade das ações corretivas ao longo do tempo;
- Identificação de dias da semana mais críticos.

Performance

A **dCalendario** apresenta baixo volume de registros e elevada frequência de leitura.

Por possuir apenas um registro por dia, mesmo um período de 30 anos resultaria em aproximadamente 11 mil linhas, quantidade pequena para o SQL Server e para o Power BI.

A utilização de **DatalD** inteiro oferece relacionamentos eficientes e excelente compressão no mecanismo VertiPaq.

Possíveis Evoluções

A dimensão poderá receber novos atributos, como:

- Indicador de feriado;
- Nome do feriado;
- Dia útil;
- Número de dia útil no mês;
- Semana comercial;
- Mês fiscal;
- Trimestre fiscal;



- Ano fiscal;
- Período promocional;
- Campanha comercial;
- Indicador de fechamento mensal;
- Indicador de inventário;
- Calendário específico da Nexus Retail.

Considerações Técnicas

A dimensão **dCalendario** é um componente fundamental da Audt Analytics Platform, pois estabelece a base temporal para todas as análises históricas.

Sua implementação centraliza os atributos de data, melhora a consistência dos relatórios, simplifica as medidas DAX e contribui para o desempenho do modelo dimensional.

Dimensão dEstado

Objetivo

A tabela **dEstado** representa a divisão político-administrativa dos estados brasileiros onde a Nexus Retail mantém operações comerciais. Sua principal finalidade é organizar geograficamente as lojas da empresa, permitindo análises estratégicas em diferentes níveis territoriais.

Esta dimensão desempenha um papel fundamental na estrutura analítica do Data Warehouse, funcionando como elo entre as regiões administrativas e as cidades onde as unidades da empresa estão localizadas.

A utilização desta dimensão permite que indicadores de desempenho sejam analisados tanto em nível estadual quanto regional, facilitando a identificação de padrões de perdas, diferenças operacionais entre estados e oportunidades de melhoria.

Além disso, sua estrutura foi projetada para suportar futuras expansões da empresa para novos estados sem necessidade de alterações estruturais no modelo de dados.

Finalidade da Dimensão

A dimensão **dEstado** possui como finalidade principal classificar as cidades e lojas segundo sua unidade federativa.



Entre as análises que poderão ser realizadas destacam-se:

- Valor total de perda por estado.
- Quantidade de ocorrências por estado.
- Comparativo de desempenho entre estados.
- Evolução mensal das perdas por unidade federativa.
- Ranking dos estados com maiores perdas.
- Participação percentual de cada estado no total de perdas.
- Comparação entre estados pertencentes à mesma região.

Essas informações servirão de apoio para decisões estratégicas relacionadas à expansão, investimentos, auditorias e ações corretivas.

Regra de Negócio

A dimensão **dEstado** deverá obedecer às seguintes regras:

- Cada estado possuirá um identificador único.
- A sigla da unidade federativa (UF) deverá ser única.
- Cada estado deverá estar obrigatoriamente vinculado a uma região.
- Um estado poderá possuir diversas cidades.
- O nome do estado deverá ser único.
- A exclusão de um estado somente será permitida caso não existam cidades vinculadas.
- Alterações na nomenclatura do estado não poderão comprometer a integridade referencial do banco.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
EstadoID	INT	Sim	PK	Identificador único do estado.
UF	CHAR(2)	Sim	UK	Sigla oficial da unidade federativa.
Estado	VARCHAR(60)	Sim	UK	Nome completo do estado.



Campo	Tipo	Obrigatório	Chave	Descrição
Regiao	VARCHAR(30)	Sim	—	Região à qual o estado pertence.

Descrição dos Campos

EstadoID

Identificador único da tabela.

Características:

- Gerado automaticamente pelo SQL Server.
- Chave Primária.
- Utilizado nos relacionamentos com a tabela **dCidade**.
- Não sofre alteração após sua criação.

UF

Armazena a sigla oficial da unidade federativa.

Exemplos:

- ES
- SP
- MG
- BA
- RJ

Será amplamente utilizada em filtros rápidos, mapas e relatórios gerenciais.

Estado

Armazena o nome completo do estado.

Exemplos:

- Espírito Santo
- São Paulo
- Minas Gerais



- Bahia
- Rio de Janeiro

RegiaoID

Campo responsável por estabelecer o relacionamento entre o estado e sua respectiva região administrativa.

Este relacionamento garante consistências nas análises geográficas e evita redundância de informações.

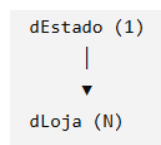
Relacionamentos

A tabela **dEstado** possui relacionamento com uma dimensão.

Com dCidade

Relacionamento:

1:N



Cada estado poderá possuir inúmeras cidade cadastradas.

Script SQL

```

CREATE TABLE dEstado
(
  EstadoID INT IDENTITY(1,1) NOT NULL,
  Estado VARCHAR(60) NOT NULL,
  UF CHAR(2) NOT NULL,
  Regiao VARCHAR(30) NOT NULL,
  CONSTRAINT PK_dEstado
    PRIMARY KEY CLUSTERED (EstadoID),
  CONSTRAINT UQ_dEstado_Estado
    UNIQUE (Estado),
  CONSTRAINT UQ_dEstado_UF
    UNIQUE (UF),
  CONSTRAINT CK_dEstado_Regiao
    CHECK (
  
```



```

        Região IN (
            'Norte',
            'Nordeste',
            'Centro-Oeste',
            'Sudeste',
            'Sul'
        )
    )
);

```

Justificativa Técnica

O uso do campo UF como atributo distinto do nome completo do estado reduz o consumo de armazenamento em consultas e melhora a experiência dos usuários finais nos dashboards.

A separação entre **UF** e **Estado** permite atender diferentes necessidades.

- Dashboards compactos utilizando apenas a sigla.
- Relatório executivos utilizando o nome completo.
- Integrações com sistemas externos que utilizam códigos oficiais.

Índices Recomendados

Além da chave primária, recomenda-se a criação dos seguintes índices:

```

CREATE UNIQUE INDEX IX_dEstado_UF
ON dEstado(UF);

```

```

CREATE INDEX IX_dEstado_Região
ON dEstado(RegiaoID);

```

Benefícios

- Melhor desempenho em consultas geográficas.
- Otimização dos filtros utilizados no Power BI.
- Aceleração dos relacionamentos com a tabela **dCidade**.
- Melhoria na performance de consultas por região.



Exemplos de Registros

EstadoID	UF	Estado	RegiaoID
1	ES	Espírito Santo	4
2	SP	São Paulo	4
3	MG	Minas Gerais	4
4	RJ	Rio de Janeiro	4
5	BA	Bahia	2

Os registros apresentados são ilustrativos e representam parte dos estados onde a Nexus Retail mantém operações.

Consultas SQL de Exemplo

Listar os estados

```
SELECT  
  
    EstadoID,  
  
    UF,  
  
    Estado  
  
FROM dEstado  
  
ORDER BY Estado;
```

Utilização no Power BI

A dimensão **dEstado** será utilizada para:

- Mapas preenchidos por UF;
- Análises comparativas entre estados;
- Segmentadores geográficos;
- Indicadores por unidade federativa;
- Ranking estaduais;
- Análise temporais por estado.

Ela também permitirá recursos de drill-down, possibilitando que o usuário navegue da região até a cidade e, posteriormente, até o nível da loja.



Possíveis Evoluções

A estrutura poderá ser expandida futuramente com a inclusão dos seguintes atributos:

- Código IBGE do estado;
- Capital;
- População;
- Área territorial;
- PIB estadual;
- Quantidade de lojas;
- Meta anual de perdas;
- Responsável regional;
- Data de início das operações da empresa no estado.

Essas informações ampliarão as possibilidades de análises estratégicas e permitirão cruzamento entre indicadores operacionais e dados socioeconômicos.

Considerações Técnicas

A dimensão dEstado desempenha papel essencial na estrutura geográfica da plataforma Audit Analytics Platform, estabelecendo a ligação entre as regiões e as cidades da operação.

Sua modelagem foi desenvolvida para garantir simplicidade, desempenho e escalabilidade, mantendo aderência às boas práticas de modelagem dimensional e preparando a solução para o crescimento da Nexus Retail.

Dimensão dLoja

Resumo da Tabela

Item	Informação
Nome da Tabela	dLoja
Tipo	Dimensão
Camada	Data Warehouse



Item	Informação
Modelo	Star Schema
Volume Estimado	128 registros (expansível)
Frequência de Atualização	Baixa
Origem dos Dados	ERP Corporativo / Cadastro Mestre de Lojas
Responsável	Engenharia de Dados
Utilizada em	Power BI, Relatórios Gerenciais, Indicadores Estratégicos
Documento Relacionado	NR-004 – Data Dictionary

Objetivo

A dimensão **dLoja** tem como objetivo armazenar todas as informações cadastrais das unidades operacionais da Nexus Retail, servindo como principal referência para a identificação das lojas nos ambientes analíticos.

Por meio desta dimensão será possível consolidar indicadores operacionais, financeiros e estratégicos em nível de loja, permitindo análises comparativas entre unidades, supervisões, gerências, estados e regiões.

A dLoja constitui um dos principais eixos analíticos do Data Warehouse, sendo utilizada em praticamente todos os dashboards da plataforma Audit Analytics Platform.

Sua estrutura foi projetada para suportar crescimento contínuo da empresa, mantendo escalabilidade, consistência e alta performance nas consultas.

Finalidade da Dimensão

A dimensão **dLoja** representa cada unidade física da Nexus Retail.

Ela permite responder perguntas estratégicas como:

- Quais lojas apresentam maior índice de perdas?
- Qual unidade possui maior valor financeiro perdido?
- Quais lojas melhoraram seus indicadores após auditorias?



- Qual supervisor administra as lojas com maior índice de perdas?
- Qual gerente regional apresenta melhor desempenho?
- Como as perdas evoluem individualmente por loja?

Além disso, a tabela permite consolidar informações em níveis superiores da hierarquia organizacional.

Regras de Negócio

A dimensão **dLoja** deverá atender às seguintes regras:

- Cada loja possua um identificador único.
- O código da loja será único em toda a empresa.
- Toda loja deverá pertencer obrigatoriamente a uma cidade.
- Toda loja deverá possuir um gerente responsável.
- Toda loja deverá possuir um supervisor responsável.
- Lojas nativas permanecerão cadastradas para preservação do histórico.
- Alterações cadastrais não poderão comprometer registros históricos da tabela fato.
- Uma loja poderá mudar de gerente ou supervisor sem perda do histórico operacional.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
LojaID	INT	Sim	PK	Identificador único da loja.
CodigoLoja	VARCHAR(10)	Sim	UK	Código interno da loja.
NomeLoja	VARCHAR(150)	Sim	—	Nome comercial da unidade.
Cidade	VARCHAR(100)	Sim	FK	Cidade onde a loja está localizada.
EstadoID	INT	Sim	FK	Estado onde a loja está localizada.



Campo	Tipo	Obrigatório	Chave	Descrição
GerenteID	INT	Sim	FK	Gerente responsável.
SupervisorID	INT	Sim	FK	Supervisor responsável.
DataInauguracao	DATE	Não	—	Data de abertura da loja.
StatusLoja	VARCHAR(20)	Sim	—	Situação da loja (Ativa, Inativa, Em Implantação).

Descrição dos Campos

LojaID

Identificador único da loja.

Características:

- Gerado automaticamente.
- Não editável.
- Utilizado em todos os relacionamentos da tabela fato.
- Chave Primária.

CodigoLoja

Código operacional utilizado pelos sistemas corporativos.

Exemplos:

- 1001
- 1025
- 1078

Será utilizado em integrações em ERP e sistemas de auditoria.

NomeLoja

Nome comercial da unidade.

Exemplos:

- Vitória Shopping



- Vila Velha Centro
- Serra Norte

CidadeID

Relaciona a loja à dimensão **dCidade**, permitindo a navegação geográfica.

GerenteID

Responsável pela gestão administrativa da unidade.

Relaciona-se à dimensão **dGerente**.

SupervisorID

Responsável pelo acompanhamento operacional da loja.

Relaciona-se à dimensão **dSupervisor**.

DataInauguracao

Permite análise como:

- Idade da loja;
- Tempo de operação;
- Perdas por tempo de funcionamento.

StatusLoja

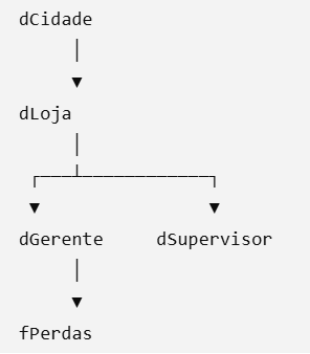
Define a situação operacional da unidade.

Valores permitidos:

- Ativa
- Inativa
- Em implementação



Relacionamento



Origem	Destino	Tipo
dCidade	dLoja	1:N
dGerente	dLoja	1:N
dSupervisor	dLoja	1:N
dLoja	fPerdas	1:N

Script SQL

```
CREATE TABLE dLoja
(
    LojaID INT IDENTITY(1,1) NOT NULL,
    CodigoLoja VARCHAR(10) NOT NULL,
    NomeLoja VARCHAR(150) NOT NULL,
    Cidade VARCHAR(100) NOT NULL,
    EstadoID INT NOT NULL
    GerenteID INT NOT NULL,
    SupervisorID INT NOT NULL,
    DataInauguracao DATE NULL,
    StatusLoja VARCHAR(20) NOT NULL,
    CONSTRAINT PK_dLoja
        PRIMARY KEY CLUSTERED (LojaID),
    CONSTRAINT UQ_dLoja_Codigo
        UNIQUE (CodigoLoja),
    CONSTRAINT FK_dLoja_Estado
```



```
FOREIGN KEY (EstadoID)

REFERENCES dEstado(EstadoID)

CONSTRAINT FK_dLoja_Gerente

FOREIGN KEY (GerenteID)

REFERENCES dGerente(GerenteID),

CONSTRAINT FK_dLoja_Supervisor

FOREIGN KEY (SupervisorID)

REFERENCES dSupervisor(SupervisorID)

);
```

Justificativa Técnica

A dimensão **dLoja** foi projetada para concentrar todos os atributos administrativos da unidade operacional, evitando redundância de informações na tabela fato.

A separação entre loja, cidade, gerente e supervisor segue os princípios da modelagem dimensional, permitindo manutenção simplificada e consultar análíticas mais eficientes.

O armazenamento do status da loja possibilita análise históricas, enquanto a data de inauguração viabiliza indicadores relacionados ao ciclo de vida das unidades.

Índices Recomendados

```
CREATE UNIQUE INDEX IX_dLoja_Codigo

ON dLoja(CodigoLoja);
```

```
CREATE INDEX IX_dLoja_Cidade

ON dLoja(CidadeID);
```

```
CREATE INDEX IX_dLoja_Gerente

ON dLoja(GerenteID);
```

```
CREATE INDEX IX_dLoja_Supervisor

ON dLoja(SupervisorID);
```

Impacto Analítico

A dimensão **dLoja** é uma das mais utilizadas em toda a plataforma analítica. Ela é a base para praticamente todos os indicadores de desempenho operacional.

Sua utilização permite:



- Ranking das lojas com maior valor de perdas.
- Comparação entre lojas de uma mesma cidade.
- Comparação entre lojas de diferentes estados.
- Avaliação de desempenho por gerente.
- Avaliação de desempenho por supervisor.
- Identificação de unidades críticas.
- Acompanhamento da evolução das perdas por loja ao longo do tempo.
- Cruzamento entre idade da loja e índice de perdas.

Essas análises subsidiam decisões relacionadas a treinamentos, auditorias, investimentos e definições de metas.

Performance

Apesar do baixo volume de registros (128 lojas na versão inicial), a dimensão **dLoja** será amplamente utilizada em junções com a tabela fato.

Para garantir desempenho adequado:

- Foram criados índices nas chaves estrangeiras;
- A chave substituta (LojaID) reduz o custo dos relacionamentos;
- A tabela apresenta baixa cardinalidade e excelente compressão no Power BI;
- O modelo favorece consultas rápidas em cenários com milhões de registros na tabela fato.

Possíveis Evoluções

A estrutura foi preparada para suportar futuras expansões, incluindo:

- Área de vendas (m²).
- Formato da loja (Shopping, Rua, Outlet).
- Tipo de operação.
- Horário de funcionamento.
- Número de colaboradores.
- Meta mensal de perdas.



- Meta anual de faturamento.
- Centro de custo.
- Data de encerramento das operações.
- Coordenadas geográficas.
- Indicador de maturidade operacional.

Considerações Técnicas

A dimensão **dLoja** constitui um dos principais pilares do modelo dimensional da Audit Analytics Platform. Sua correta implementação garante a integridade das análises operacionais e possibilita que os dashboards reflitam com precisão o desempenho individual de cada unidade da Nexus Retail.

Além de atender às necessidades atuais da empresa, sua estrutura foi planejada para suportar crescimento, reorganizações administrativas e novas demandas analíticas, preservando a consistência e a escalabilidade da solução.

Dimensão dSupervisor

Resumo da Tabela

Item	Informação
Nome da Tabela	dSupervisor
Tipo	Dimensão
Camada	Data Warehouse
Modelo	Star Schema
Volume Estimado	Aproximadamente 30 registros
Frequência de Atualização	Baixa
Origem dos Dados	Cadastro Organizacional da Nexus Retail
Responsável	Engenharia de Dados

Item	Informação
Utilizada em	Power BI, Dashboards Gerenciais e Relatórios Analíticos
Documento Relacionado	NR-004 – Data Dictionary

Objetivo

A dimensão **dSupervisor** tem como objetivo armazenar as informações dos supervisores responsáveis pelas unidades operacionais da Nexus Retail.

Sua principal finalidade é permitir que cada loja seja associada ao respectivo supervisor, possibilitando análises de desempenho operacional, acompanhamento dos indicadores de perdas e avaliação da efetividade das ações de auditoria.

A utilização desta dimensão evita redundância de informações na tabela de lojas e facilita futuras alterações na estrutura organizacional da empresa.

Finalidade da Dimensão

A dimensão **dSupervisor** representa a nível de supervisão operacional da Nexus Retail.

Sua utilização permite consolidar indicadores gerenciais por supervisor, possibilitando análise como:

- Comparação de perdas entre supervisores.
- Ranking dos supervisores com melhor desempenho.
- Evolução mensal das perdas por supervisor.
- Quantidade de lojas supervisionadas.
- Valor financeiro das perdas por equipe.
- Eficiência das auditorias realizadas.

Essas informações auxiliam a gestão na tomada de decisões relacionadas à prevenção de perdas e ao desempenho das equipes.

Regras de Negócio

A dimensão **dSupervisor** deverá atender às seguintes regras:



- Cada supervisor possuirá um identificador único.
- Um supervisor poderá ser responsável por uma ou mais lojas.
- Toda loja deverá possuir um supervisor vinculado.
- Supervisores inativos permanecerão cadastrados para preservar o histórico das análises.
- Alterações cadastrais não deverão comprometer os registros históricos da tabela fato.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
SupervisorID	INT	Sim	PK	Identificador único do supervisor.
NomeSupervisor	VARCHAR(120)	Sim	—	Nome do supervisor.
StatusSupervisor	VARCHAR(20)	Sim	—	Situação do supervisor (Ativo ou Inativo).

Descrição dos Campos

SupervisorID

Identificador único do supervisor.

Características:

- Gerado automaticamente pelo SQL Server.
- Chave Primária.
- Utilizado nos relacionamentos com o dimensão **dLoja**.

NomeSupervisor

Armazena o nome do supervisor responsável pelas lojas.

Exemplos:

- Carlos Eduardo Martins
- Juliana Ferreira



- André Luiz Souza

Este campo será utilizado em filtros, segmentações e relatórios analíticos.

StatusSupervisor

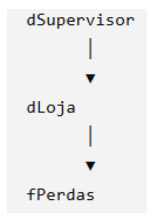
Indica a situação atual do supervisor.

Valores permitidos:

- Ativo
- Inativo

A manutenção dos supervisores inativos preserva a integridade histórica das análises.

Relacionamentos



Cardinalidade

Origem	Destino	Tipo
dSupervisor	dLoja	1:N
dLoja	fPerdas	1:N

Script SQL

```
CREATE TABLE dSupervisor
(
    SupervisorID INT IDENTITY(1,1) NOT NULL,
    NomeSupervisor VARCHAR(120) NOT NULL,
    StatusSupervisor VARCHAR(20) NOT NULL DEFAULT('Ativo'),
    CONSTRAINT PK_dSupervisor
        PRIMARY KEY CLUSTERED (SupervisorID)
);
```



Justificativa Técnica

A separação da entidade **Supervisor** em uma dimensão específica reduz redundâncias e melhora a organização do modelo dimensional.

Essa abordagem permite que alterações na estrutura organizacional da empresa sejam realizadas sem impactar os registros históricos da tabela fato, preservando a consistência das análises ao longo do tempo.

Além disso, a utilização de uma chave substituta (SupervisorID) simplifica os relacionamentos entre as tabelas e melhora o desempenho das consultas analíticas.

Índices Recomendados

```
CREATE INDEX IX_dSupervisor_Nome
ON dSupervisor (NomeSupervisor);
```

Exemplos de Registros

SupervisorID	NomeSupervisor	StatusSupervisor
1	Carlos Eduardo Martins	Ativo
2	Juliana Ferreira	Ativo
3	André Luiz Souza	Ativo
4	Mariana Costa	Inativo

Consultas SQL de Exemplo

Supervisores ativos

```
SELECT
    SupervisorID,
    NomeSupervisor
FROM dSupervisor
WHERE StatusSupervisor = 'Ativo'
ORDER BY NomeSupervisor;
```

Quantidade de lojas por supervisor

```
SELECT
```



```
s.NomeSupervisor,  
  
COUNT(l.LojaID) AS TotalLojas  
  
FROM dSupervisor s  
  
LEFT JOIN dLoja l  
  
ON s.SupervisorID = l.SupervisorID  
  
GROUP BY s.NomeSupervisor  
  
ORDER BY TotalLojas DESC;
```

Utilização no Power BI

A dimensão **dSupervisor** será utilizada para:

- Segmentadores por supervisor.
- Rankings de perdas por supervisor.
- Comparativos entre equipes.
- Indicadores de desempenho operacional.
- Dashboards de gestão regional.
- Análises temporais por responsável.

Impacto Analítico

A dimensão **dSupervisor** permite analisar o desempenho das equipes sob responsabilidade de cada supervisor, identificando padrões de perdas e oportunidades de melhoria.

Por meio dessa dimensão será possível:

- Comparar resultados entre supervisores;
- Identificar equipes com melhor desempenho;
- Acompanhar a evolução dos indicadores ao longo do tempo;
- Avaliar o impacto das auditorias realizadas;
- Subsidiar decisões relacionadas ao treinamento e à gestão operacional.

Performance

A dimensão apresenta baixo volume de registros e alta frequência de leitura, sendo otimizada para consultas analíticas.



A utilização de chave substituta e índice sobre o nome do supervisor proporciona consultas rápidas e eficientes, especialmente em cenários com grandes volumes de dados na tabela fato.

Possíveis Evoluções

A estrutura foi planejada para permitir futuras expansões, como a inclusão de:

- Código interno do supervisor.
- Região de atuação.
- Data de início da supervisão.
- Data de término da supervisão.
- E0mail corporativo.
- Indicadores individuais de desempenho.
- Histórico de movimentações entre regiões.

Considerações Técnicas

A dimensão **dSupervisor** representa um componente essencial da estrutura organizacional da Audit Analytics Platform. Sua implementação garante rastreabilidade da gestão das lojas, preservação do histórico analítico e suporte à construção de indicadores estratégicos relacionados ao desempenho operacional.

Dimensão dGerente

Resumo da Tabela

Item	Informação
Nome da Tabela	dGerente
Tipo	Dimensão
Camada	Data Warehouse
Modelo	Star Schema
Volume Estimado	Aproximadamente 15 registros
Frequência de Atualização	Baixa



Item	Informação
Origem dos Dados	Cadastro Organizacional da Nexus Retail
Responsável	Engenharia de Dados
Utilizada em	Power BI, Dashboards Executivos e Relatórios Gerenciais
Documento Relacionado	NR-004 – Data Dictionary

Objetivo

A dimensão **dGerente** tem como objetivo armazenar as informações dos gerentes responsáveis pelas unidades operacionais da Nexus Retail.

Sua utilização permite consolidar os resultados das lojas sob responsabilidade de cada gerente, oferecendo uma visão estratégica dos indicadores de perdas, desempenho operacional e efetividade das ações de prevenção.

Ao manter essas informações em uma dimensão específica, garante-se maior organização do modelo dimensional, facilidade de manutenção e flexibilidade para futuras alterações na estrutura administrativa da empresa.

Finalidade da Dimensão

A dimensão **dGerente** representa o nível gerencial da estrutura organizacional na Nexus Retail.

Ela permite acompanhar indicadores como:

- Valor total de perdas por gerência.
- Quantidade de lojas administradas por cada gerente.
- Evolução mensal dos indicadores.
- Comparação de desempenho entre gerências.
- Eficiência das ações de prevenção de perdas.
- Distribuição das perdas por estado e por loja.

Essa análises apoiam decisões estratégicas relacionadas ao desempenho das operações e ao planejamento das ações corporativas.



Regras de Negócio

A dimensão **dGerente** deverá atender às seguintes regras:

- Cada gerente possuirá um identificador único.
- Um gerente poderá ser responsável por uma ou mais lojas.
- Toda loja deverá possuir um gerente vinculado.
- Gerentes inativos permanecerão cadastrados para preservação do histórico analítico.
- Alterações cadastrais não poderão comprometer os registros históricos da tabela fato.

Estrutura de Negócio

A dimensão **dGerente** deverá atender às seguintes regras:

- Cada gerente possuirá um identificador único.
- Um gerente poderá ser responsável por uma ou mais lojas.
- Toda loja deverá possuir um gerente vinculado.
- Gerentes inativos permanecerão cadastrados para preservação dos histórico analítico.
- Alterações cadastrais não poderão comprometer os registros históricos da tabela fato.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
GerenteID	INT	Sim	PK	Identificador único do gerente.
NomeGerente	VARCHAR(120)	Sim	—	Nome do gerente.
StatusGerente	VARCHAR(20)	Sim	—	Situação do gerente (Ativo ou Inativo).

Descrição dos Campos

GerenteID

Identificador único do gerente.



Características:

- Gerado automaticamente pelo SQL Server.
- Chave Primária.
- Utilizado nos relacionamentos com a dimensão **dLoja**.

NomeGerente

Armazena o nome completo do gerente pelas unidades operacionais.

Exemplos:

- Ricardo Almeida
- Fernanda Souza
- Marcelo Oliveira

Esse campo será utilizado em filtros, segmentações e relatórios gerenciais.

StatusGerente

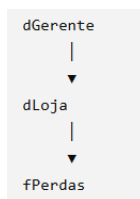
Indica a situação do gerente na organização.

Valores permitidos:

- Ativo
- Inativo

A manutenção dos gerentes inativos garante a preservação do histórico das análises e evita inconsistências nos dados históricos.

Relacionamentos



Cardinalidade

Origem	Destino	Tipo
dGerente	dLoja	1:N
dLoja	fPerdas	1:N



Script SQL

```
CREATE TABLE dGerente
(
    GerenteID INT IDENTITY(1,1) NOT NULL,
    NomeGerente VARCHAR(120) NOT NULL,
    StatusGerente VARCHAR(20) NOT NULL DEFAULT('Ativo'),
    CONSTRAINT PK_dGerente
        PRIMARY KEY CLUSTERED (GerenteID)
);
```

Justificativa Técnica

A criação da dimensão **dGerente** possibilita representar a estrutura gerencial da Nexus Retail de forma organizada e independente das demais dimensões.

Essa abordagem reduz redundâncias, facilita a manutenção dos dados organizacionais e preserva o histórico das análises mesmo diante de elterações na estrutura administrativa da empresa.

Além disso, o uso de uma chave substituta (GerenteID) melhora a eficiência dos relacionamentos com a dimensão **dLoja** e contribui par ao desempenho das consultas analíticas.

Dimensão dCategoria

Resumo da Tabela

Item	Informação
Nome da Tabela	dCategoria
Tipo	Dimensão
Camada	Data Warehouse
Modelo	Star Schema
Volume Estimado	Aproximadamente 80 registros
Frequência de Atualização	Baixa

Item	Informação
Origem dos Dados	ERP Corporativo / Cadastro Mestre de Produtos
Responsável	Engenharia de Dados
Utilizada em	Power BI, Dashboards Executivos, Relatórios Analíticos
Documento Relacionado	NR-004 – Data Dictionary

Objetivo

A dimensão **dCategoria** representa o segundo nível da hierarquia comercial dos produtos da Nexus Retail, agrupando itens semelhantes dentro de um mesmo departamento.

Seu principal objetivo é permitir análises mais detalhadas das perdas operacionais, possibilitando identificar quais categorias apresentam maior impacto financeiro, maior frequência de ocorrências ou maior necessidade de ações corretivas.

Ao organizar os produtos em categorias, a plataforma oferece uma visão intermediária entre os departamentos e os produtos individuais, facilitando a identificação de padrões de comportamento e tendências de perdas.

Finalidade da Dimensão

A dimensão **dCategoria** tem como finalidade estruturar o cadastro de produtos em grupos específicos dentro de cada departamento.

Exemplos de categorias:

Departamento	Categoria
Smartphones	Smartphones Android
Smartphones	Smartphones iPhone
Informática	Notebooks
Informática	Monitores
Áudio	Fones Bluetooth



Departamento	Categoria
Áudio	Caixas de Som
Games	Consoles
Games	Jogos
Acessórios	Carregadores
Acessórios	Cabos USB

Essa classificação permite análise mais detalhadas sem chegar ao nível individual do produto.

Regras de Negócio

A dimensão **dCategoria** deverá obedecer às seguintes regras:

- Cada categoria possuirá um identificador único.
- Toda categoria deverá pertencer obrigatoriamente a um departamento.
- Um departamento poderá possuir diversas categorias.
- O nome da categoria poderá repetir-se em departamentos diferentes, desde que a combinação Departamento + Categoria seja única.
- Uma categoria poderá possuir inúmeros produtos cadastrados.
- Não será permitida a exclusão de categorias utilizadas por produtos ativos.
- Categorias inativas permanecerão cadastradas para preservação do histórico analítico.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
CategoriaID	INT	Sim	PK	Identificador único da categoria.
DepartamentoID	INT	Sim	FK	Departamento ao qual pertence.
NomeCategoria	VARCHAR(100)	Sim	—	Nome da categoria.
Ativo	BIT	Sim	—	Situação da categoria.



Descrição dos Campos

CategoriaID

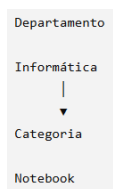
Identificador único da categoria.

- Chave Primária.
- Gerado automaticamente.
- Utilizado pela dimensão **dProduto**.

DepartamentoID

Relaciona a categoria ao seu departamento.

Exemplo:



NomeCategoria

Nome utilizado na classificação comercial.

Exemplos:

- Smartphones Android
- Smartphones iPhone
- Monitores
- Carregadores
- Smartwatch

VALores

Ativo

Campo responsável por indicar se a categoria permanece disponível para utilização.

Valores:

- 1 = Ativa



- 0 = Inativa

Relacionamentos



Cardinalidade

Origem	Destino	Tipo
dDepartamento	dCategoria	1:N
dCategoria	dProduto	1:N

Script SQL

```
CREATE TABLE dCategoria
(
    CategoriaID INT IDENTITY(1,1) NOT NULL,
    DepartamentoID INT NOT NULL,
    NomeCategoria VARCHAR(100) NOT NULL,
    Ativo BIT NOT NULL DEFAULT(1),
    CONSTRAINT PK_dCategoria
        PRIMARY KEY CLUSTERED (CategoriaID),
    CONSTRAINT FK_dCategoria_Departamento
        FOREIGN KEY (DepartamentoID)
            REFERENCES dDepartamento(DepartamentoID),
    CONSTRAINT UQ_dCategoria
        UNIQUE (DepartamentoID, NomeCategoria)
);
```

Justificativa Técnica

A separação entre departamentos e categoria reduz redundâncias e trona o modelo mais flexível.

Essa estrutura permite reorganizações comerciais sem necessidade de alterar os registros históricos da tabela fato.



O uso da chave substituta (CategoriaID) simplifica as relacionamentos e melhora o desempenho das consultas analíticas.

Índices Recomendados

CREATE INDEX IX_dCategoria_Departamento
ON dCategoria (DepartamentoID);

CREATE INDEX IX_dCategoria_Nome
ON dCategoria (NomeCategoria);

Exemplos de Registros

CategoriaID	DepartamentoID	NomeCategoria	Ativo
1	1	Smartphones Android	1
2	1	Smartphones iPhone	1
3	2	Notebooks	1
4	2	Monitores	1
5	3	Consoles	1
6	4	Fones Bluetooth	1
7	5	Carregadores	1

Consultas SQL de Exemplo

Listar categorias por departamento

```
SELECT
    d.NomeDepartamento,
    c.NomeCategoria
FROM dCategoria c
INNER JOIN dDepartamento d
    ON c.DepartamentoID = d.DepartamentoID
ORDER BY
    d.NomeDepartamento,
    c.NomeCategoria;
```

Quantidade de produtos por categoria

```
SELECT
```



```
c.NomeCategoria,  
  
COUNT(p.ProdutoID) AS TotalProdutos  
  
FROM dCategoria c  
  
LEFT JOIN dProduto p  
  
ON c.CategoriaID = p.CategoriaID  
  
GROUP BY c.NomeCategoria  
  
ORDER BY TotalProdutos DESC;
```

Utilização no Power BI

A dimensão **dCategoria** será utilizada em:

- Segmentadores por categoria.
- Gráficos de barras por categoria.
- Pareto de perdas por categoria.
- Heatmaps de perdas.
- Rankings de categorias críticas.
- Drill-down Departamento → Categoria → Produto.

Impacto Analítico

A dimensão **dCategoria** aproxima a análise da realidade operacional da empresa, permitindo identificar grupos de produtos que exigem maior atenção.

Com ela será possível:

- Identificar categorias com maior índice de perdas;
- Avaliar tendências de perdas ao longo do tempo;
- Comparar categorias entre diferentes departamentos;
- Medir a efetividade de ações corretivas;
- Apoiar decisões sobre treinamento, exposição de produtos e processo de armazenamento.

Essas análises auxiliam a diretoria na priorização de investimentos e na definição de planos de ação mais assertivos.

Performance

A dimensão **dCategoria** apresenta baixo volume de registros e alta frequência de leitura.



As principais otimizações adotadas incluem:

- Utilização de chave substituta (CategoricalID);
- Índice na chave estrangeira (DepartamentoID);
- Índice para pesquisas por nome;
- Baixa cardinalidade, favorecendo compressão no Power BI;
- Relacionamentos simples (1:N), reduzindo o custo das consultas.

Possíveis Evoluções

A estrutura foi planejada para suportar futuras necessidades, permitindo a inclusão de atributos como:

- Código da categoria no ERP.
- Categoria fiscal.
- Categoria logística.
- Categoria marketing
- Responsável pela categoria.
- Meta anual de perdas.
- Meta de faturamento.
- Curva ABC.
- Criticidade operacional.
- Percentual médio de margem.
- Data de criação.
- Data de descontinuação.

Considerações Técnicas

A dimensão **dCategoria** é um elemento fundamental da hierarquia comercial da AUdit Analytics Platform, permitindo organizar os produtos em níveis intermediários entre departamentos e itens individuais.

Sua implementação favorece análises detalhadas, melhora a qualidade dos indicadores gerenciais e oferece uma estrutura escalável para futuras expansões do catálogo da Nexus Retail.



Dimensão dProduto

Resumo da Tabela

Item	Informação
Nome da Tabela	dProduto
Tipo	Dimensão
Camada	Data Warehouse
Modelo	Star Schema
Volume Estimado	Aproximadamente 80.000 registros
Frequência de Atualização	Diária
Origem dos Dados	ERP Corporativo da Nexus Retail
Responsável	Engenharia de Dados
Utilizada em	Dashboards Power BI, Relatórios Gerenciais, Indicadores Estratégicos
Documento Relacionado	NR-004 – Data Dictionary

Objetivo

A dimensão **dProduto** tem como finalidade armazenar todas as informações cadastrais dos produtos comercializados pela Nexus Retail, servindo como referência para os processos analíticos relacionados à auditoria de perdas.

Ela representa uma das principais dimensões do modelo Star Schema, permitindo associar cada registro de perda ao respectivo produto e possibilitando análises detalhadas sobre desempenho, incidência de perdas, comportamento por categoria, marca e fornecedor.

A centralização dessas informações em uma dimensão específica reduz redundâncias, facilita a manutenção do cadastro e melhora significativamente o desempenho das consultas analíticas.



Papel da Dimensão no Modelo Star Schema

A dimensão **dProduto** desempenha papel central na estrutura dimensional da plataforma Audit Analytics Platform.

Ela funciona como ponto de ligação entre os dados cadastrais dos produtos e a tabela fato **fPerdas**, permitindo que os indicadores sejam analisados sob diferentes perspectivas de negócio.

Por meio dessa dimensão será possível responder perguntas como:

- Quais produtos apresentam maior índice de perdas?
- Qual marca gera maior impacto financeiro?
- Quais categorias concentram mais ocorrências?
- Quais fornecedores possuem maior volume de perdas?
- Qual valor perdido por produto em determinado período?
- Como as perdas evoluem ao longo do tempo para um produto específico?

Essa flexibilidade torna a dimensão indispensável para a construção de dashboards estratégicos e para a tomada de decisão baseada em dados.

Regras de Negócio

A dimensão **dProduto** deverá obedecer às seguintes regras:

- Cada produto possuirá um identificador único (ProdutoID).
- O código do produto deverá ser único dentro do ambiente analítico.
- Todo produto deverá estar vinculado a uma categoria válida.
- Todo produto deverá possuir uma marca cadastrada.
- Todo produto deverá possuir um fornecedor associado.
- Produtos inativos permanecerão cadastrados para preservar o histórico das análises.
- Alterações cadastrais não deverão modificar registros históricos da tabela fato.
- O valor de venda deverá representar o preço vigente no momento da sincronização do cadastro.



Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
ProdutoID	INT	Sim	PK	Identificador único do produto.
CodigoProduto	VARCHAR(30)	Sim	—	Código interno utilizado pelo ERP.
NomeProduto	VARCHAR(150)	Sim	—	Nome comercial do produto.
CategoriaID	INT	Sim	FK	Referência à dimensão dCategoria.
ValorVenda	DECIMAL(18,2)	Sim	—	Valor de venda vigente do produto.
Ativo	BIT	Sim	—	Situação do produto (ativo ou inativo).

Descrição dos Campos

ProdutoID

Identificador único do produto dentro do Data Warehouse.

Características:

- Gerado automaticamente pelo SQL Server.
- Chave Primária da dimensão.
- Utilizado como chave estrangeira na tabela dato **fPerdas**.

CodigoProduto

Código interno utilizado pelo ERP da empresa para identificar cada item comercializado.

Características:

- Deve ser único.
- Utilizado em integrações entre sistemas.



- Permite rastrear o produto desde a origem operacional até os relatórios analíticos.

Exemplos:

- IPH16-128-BLK
- SAM-S24-256
- JBL-FLIP6

NomeProduto

Armazena a descrição comercial do produto.

Exemplos:

- iPhone 16 128 GB Preto
- Samsung Galaxy S24 256 GB
- JBL Flip 6 Azul

Esse atributo será amplamente utilizado em filtros, tabelas, gráficos e análises detalhadas no Power BI.

CategoriaID

Identifica a categoria à qual o produto pertence.

Esse relacionamento permite consolidar indicadores por grupos de produtos e realizar análises comparativas entre categorias.

ValorVenda

Representa o valor de venda vigente do produto no momento da carga do Data Warehouse.

Esse campo é utilizado para:

- Análises financeiras;
- Estimativas de impacto das perdas;
- Construção de indicadores monetários;
- Comparação entre perdas físicas e financeiras.

Ativo

Indica a situação cadastral do produto.



Valores possíveis:

- 1 – Produto Ativo
- 0 – Produto Inativo

Produtos inativos permanecem cadastrados para preservar o histórico das análises.

Relacionamentos

A dimensão **dProduto** possui relacionamento com diversas tabelas do modelo dimensional, conforme descrito abaixo:

Origem	Destino	Tipo
dCategoria	dProduto	1:N
dMarca	dProduto	1:N
dFornecedor	dProduto	1:N
dProduto	fPerdas	1:N

Esses relacionamentos permitem que os indicadores da tabela fato sejam analisados sob diferentes perspectivas de negócio, mantendo a integridade referencial do Data Warehouse.

Script SQL

A implementação física da dimensão **dProduto** foi projetada para armazenar os dados cadastrais dos produtos de forma normalizada, preservando a integridade referencial com as demais dimensões do modelo dimensional.

A tabela utiliza uma chave primária substitua (ProdutoID) e mantém relacionamentos com as dimensões **dCategoria**, **dMarca** e **dFornecedor**, garantindo consistência dos dados e facilitando consultas analíticas.

```
CREATE TABLE dProduto
(
    ProdutoID INT IDENTITY(1,1) NOT NULL,
    CodigoProduto VARCHAR(30) NOT NULL,
    NomeProduto VARCHAR(150) NOT NULL,
    CategoriaID INT NOT NULL,
```



```

ValorVenda DECIMAL(18,2) NOT NULL,

Ativo BIT NOT NULL

CONSTRAINT DF_dProduto_Ativo DEFAULT (1),

CONSTRAINT PK_dProduto

PRIMARY KEY CLUSTERED (ProdutoID),

CONSTRAINT UQ_dProduto_Codigo

UNIQUE (CodigoProduto),

CONSTRAINT FK_dProduto_Categoria

FOREIGN KEY (CategorialID)

REFERENCES dCategoria(CategorialID),

CONSTRAINT CK_dProduto_ValorVenda

CHECK (ValorVenda >= 0)

);

```

Constraints

A tabela **dProduto** utiliza restrições para garantir a integridade dos dados armazenados.

Chave Primária

Constraint	Campo	Finalidade
------------	-------	------------

PK_dProduto	ProdutoID	Identificar unicamente cada produto da dimensão.
-------------	-----------	--

Chaves Estrangeiras

Constraint	Origem	Destino
FK_dProduto_Categoria	CategorialID	dCategoria

Essas restrições impedem que produtos sejam cadastrados com referências inexistentes, garantindo consistência entre as dimensões.

Restrições de Integridade

Além das chaves, aplicam-se as seguintes regras:

- O campo **CodigoProduto** não poderá ser nulo.
- O campo **NomeProduto** não poderá ser nulo.



- Todo produto deverá possuir categoria válida.
- Todo produto deverá possuir marca válida.
- Todo produto deverá possuir fornecedor válido.
- O campo **ValorVenda** deverá ser maior ou igual a zero.
- O campo **Ativo** aceitará apenas os valores **0** ou **1**.

Índices Recomendados

Para melhorar o desempenho das consultas analíticas, recomenda-se a criação dos seguintes índices.

Índice por Código do Produto

```
CREATE UNIQUE INDEX IX_dProduto_Codigo
ON dProduto (CodigoProduto);
```

Finalidade

- Evitar duplicidade de códigos.
- Acelerar integrações como o ERP.
- Melhorar pesquisas por código.

Índice por Nome

```
CREATE INDEX IX_dProduto_Nome
ON dProduto (NomeProduto);
```

Finalidade

- Melhorar pesquisas textuais.
- Otimizar filtros no Power BI.

Índice por Categoria

```
CREATE INDEX IX_dProduto_Categoria
ON dProduto (CategorialID);
```

Finalidade

- Acelerar agrupamentos por categoria.
- Melhorar consultas analíticas.



Exemplos de Registros

ProdutoID	Código	Produto	Categoria	Marca	Fornecedor	Valor Venda	Ativo
1	CEL0001	Smartphone Alpha X	Smartphones	AlphaTech	Alpha Distribuição	R\$ 3.499,90	Sim
2	TAB0005	Tablet Vision 10	Tablets	Vision	Tech Supply	R\$ 1.899,00	Sim
3	FON0012	Fone Bluetooth AirSound	Áudio	AirSound	Music Import	R\$ 399,90	Sim
4	CAB0101	Cabo USB-C 2m	Acessórios	Connect	Connect Brasil	R\$ 49,90	Sim

Observação: OS registros acima são fictícios e têm finalidade exclusivamente ilustrativa.

Consultas SQL de Exemplo

Listar todos os produtos ativos

```
SELECT
    ProdutoID,
    CodigoProduto,
    NomeProduto,
    ValorVenda
FROM dProduto
WHERE Ativo = 1
ORDER BY NomeProduto;
```

Produtos por categoria

```
SELECT
    c.NomeCategoria,
    COUNT(p.ProdutoID) AS TotalProdutos
FROM dCategoria c
LEFT JOIN dProduto p
    ON c.CategoriaID = p.CategoriaID
GROUP BY c.NomeCategoria
ORDER BY TotalProdutos DESC;
```


Valor médio dos produtos

SELECT

AVG(ValorVenda) AS ValorMedio

FROM dProduto;

Utilização da Dimensão dProduto no Power BI

A dimensão **dProduto** desempenha papel fundamental na construção dos dashboards da Audit Anaçytics Platform, fornecendo os atributos necessários para análises detalhadas dos produtos envolvidos nas ocorrências de perdas.

No Power BI, essa dimensão será utilizada para enriquecer os dados provenientes da tabela fato **fPerdas**, permitindo análises sob diferentes perspectivas e facilitando a identificação de padrões de comportamento.

Entre as principais aplicações destacam-se:

- Segmentação por produto;
- Segmentação por categoria;
- Pesquisa por código do produto;
- Rankings dos produtos com maior quantidade de perdas;
- Rankings dos produtos com maior impacto financeiro;
- Comparação entre categorias de produtos;
- Evolução histórica das perdas por produto;
- Dashboards operacionais para auditoria.

A utilização da dimensão reduz a necessidade de repetição de informações na tabela fato, contribuindo para um modelo mais eficiente e de melhor desempenho.

Indicadores Utilizados

Indicadores Operacionais

- Quantidade de perdas por produto;
- Quantidade de ocorrências por produto;
- Produtos com maior reincidência de perdas;
- Produtos mais auditados.



Indicadores Financeiros

- Valor total perdido por produto;
- Valor médio das perdas;
- Participação percentual de cada produto no total das perdas;
- Impacto financeiro por categoria.

Indicadores Gerenciais

- Ranking dos produtos com maior prejuízo.
- Ranking das categorias com maior índice de perdas;
- Comparativo entre lojas utilizando os mesmos produtos;
- Evolução mensal das perdas por produto.

Esses indicadores subsidiam a tomada de decisão das áreas de Auditoria, Operações e Controladoria.

Estratégia de Atualização da Dimensão

A atualização da dimensão **dProduto** ocorrerá por meio do processo de ETL da plataforma.

Durante cada carga serão executadas as seguintes etapas:

1. Extração do cadastro de produtos dos sistemas de origem.
2. Validação da integridade dos registros.
3. Verificação dos produtos já existentes.
4. Inclusão de novos produtos.
5. Atualização dos atributos alterados.
6. Validação dos relacionamentos com a dimensão **dCategoria**.
7. Registro das alterações no ambiente analítico.

Essa estratégia garante que o Data Warehouse permaneça sincronizado com os sistemas operacionais, preservando a consistência dos dados utilizados nas análises.



Performance

A dimensão **dProduto** apresenta maior volume de registros quando comparada às demais dimensões do projeto, exigindo cuidados específicos relacionados ao desempenho.

Para garantir consultas eficientes, foram adotadas as seguintes estratégias:

- Utilização de chave primária substituta (ProdutoID);
- Índice sobre o código do produto;
- Índice sobre a categoria do produto;
- Índice para pesquisar por nome;
- Relacionamento simplificado com a tabela fato.

Essas práticas reduzem o tempo de resposta das consultas e melhoram o desempenho dos dashboards desenvolvidos no Power BI.

Possíveis Evoluções

Embora a estrutura atual atenda aos requisitos da Audit ANalytics Platform, futuras versões poderão incorporar novos atributos para ampliar a capacidade analíticas da dimensão.

Entre as possíveis evoluções destacam-se:

- Código de barras do produto;
- Unidade de medida;
- Data de lançamentos;
- Data de descontinuação;
- Fabricante;
- Linha de produtos;
- Peso;
- Dimensões físicas;
- Valor de custo;
- Margem de contribuição;
- Centro de distribuição responsável.



A inclusão desses atributos dependerá das necessidades do negócio e da disponibilidade das informações nos sistemas de origem.

Considerações Técnicas

A dimensão **dProduto** constitui um dos principais componentes do modelo dimensional da Audit Analytics Platform.

Sua implementação permite centralizar as informações cadastrais dos produtos, garantindo padronização, integridade e consistência dos dados utilizados nas análises.

Ao manter os atributos dos produtos separados da tabela fato, o modelo reduz redundâncias, melhora o desempenho das consultas e facilita a manutenção do ambiente analítico.

Além disso, a adoção de uma estrutura dimensional favorece a escalabilidade da solução, permitindo a inclusão de novos atributos e funcionalidades sem comprometer a arquitetura existente.

Tabela Fato fPerdas

Resumo da Tabela

Item	Informação
Nome da Tabela	fPerdas
Tipo	Tabela Fato
Camada	Data Warehouse
Modelo	Star Schema
Granularidade	Uma linha por ocorrência de perda, por loja, data e produto
Volume Estimado	Crescimento contínuo, com potencial para milhões de registros
Frequência de Atualização	Diária



Item	Informação
Origem dos Dados	Sistema de Auditoria, ERP e bases operacionais
Responsável	Engenharia de Dados
Utilizada em	Power BI, dashboards executivos e relatórios gerenciais
Documento Relacionado	NR-005 – Data Model Specification

Objetivo

A tabela fato fPerdas constitui o núcleo do modelo dimensional da Audit Analytics Platform.

Sua finalidade é armazenar cada ocorrência de perda registrada pela Nexus Retail, preservando o contexto temporal, operacional, organizacional e comercial do evento.

A tabela concentra as métricas quantitativas utilizadas na análise de perdas e estabelece relacionamento com as dimensões responsáveis pela descrição de cada ocorrência.

Papel da Tabela Fato

Enquanto as dimensões armazenam atributos descritivos, a tabela fato armazena os eventos mensuráveis do processo de auditoria.

A fPerdas permitirá responder perguntas como:

- Qual foi o valor total das perdas?
- Qual loja apresenta maior prejuízo?
- Quais produtos concentram mais ocorrências?
- Qual categoria apresenta maior impacto financeiro?
- Qual gerente possui maior valor de perdas sob sua responsabilidade?
- Qual supervisor acompanha as unidades mais críticas?
- Como as perdas evoluíram ao longo dos meses?
- Qual tipo de perda ocorre com maior frequência?



Granularidade

A granularidade definida para a tabela é: uma linha para cada ocorrência de perda registrada em uma loja, em determinada data e para determinado produto.

Cada ocorrência deverá ser armazenada individualmente, mesmo quando envolver o mesmo produto, a mesma loja ou a mesma data de outra ocorrência. Essa definição permite análises detalhadas e agregações por período, loja, produto, categoria, supervisor, gerente e tipo de perda.

Estrutura da Tabela

Campo	Tipo	Obrigatório	Chave	Descrição
PerdaID	BIGINT	Sim	PK	Identificador único da ocorrência de perda.
DataID	INT	Sim	FK	Data relacionada à ocorrência.
LojaID	INT	Sim	FK	Loja em que a perda foi registrada.
ProdutoID	INT	Sim	FK	Produto envolvido na ocorrência.
CategoriaID	INT	Sim	FK	Categoria do produto no momento da ocorrência.
SupervisorID	INT	Sim	FK	Supervisor responsável no momento da ocorrência.
GerenteID	INT	Sim	FK	Gerente responsável no momento da ocorrência.
Quantidade	INT	Sim	—	Quantidade de unidades perdidas.
ValorPerda	DECIMAL(18,2)	Sim	—	Valor financeiro total da ocorrência.
TipoPerda	VARCHAR(50)	Sim	—	Classificação da perda registrada.

Descrição dos Campos

PerdaID

Identificador único de cada ocorrência. É gerado automaticamente pelo SQL Server, utiliza o tipo BIGINT e constitui a chave primária da tabela.

DataID

Chave estrangeira relacionada à dimensão dCalendario. Permite analisar a ocorrência por dia, semana, mês, trimestre, semestre e ano.

LojaID

Chave estrangeira relacionada à dimensão dLoja. Identifica a unidade da Nexus Retail em que a ocorrência foi registrada.

ProdutoID

Chave estrangeira relacionada à dimensão dProduto. Identifica o produto envolvido no evento de perda.

CategoriaID

Chave estrangeira relacionada à dimensão dCategoria. Preserva diretamente a categoria associada ao evento no momento da ocorrência.

SupervisorID

Chave estrangeira relacionada à dimensão dSupervisor. Representa o supervisor responsável no momento do evento.

GerenteID

Chave estrangeira relacionada à dimensão dGerente. Representa o gerente responsável no momento do evento.

Quantidade

Armazena a quantidade de unidades envolvidas na ocorrência e deverá possuir valor maior que zero.

ValorPerda

Armazena o valor financeiro total da ocorrência, utilizando DECIMAL(18,2) para assegurar precisão monetária.

TipoPerda



Armazena a classificação da perda, como Furto, Avaria, Quebra, Divergência de Estoque, Erro Operacional ou Extravio.

Métricas Armazenadas

- Quantidade perdida: total físico de unidades envolvidas nas ocorrências.
- Valor da perda: impacto financeiro total das ocorrências.
- Quantidade de ocorrências: calculada pela contagem dos registros ou do PerdaID.

Relacionamentos

A tabela fPerdas possui relacionamentos diretos do tipo um para muitos com as dimensões do modelo.

Origem	Destino	Cardinalidade
dCalendario	fPerdas	1:N
dLoja	fPerdas	1:N
dProduto	fPerdas	1:N
dCategoria	fPerdas	1:N
dSupervisor	fPerdas	1:N
dGerente	fPerdas	1:N

No Power BI, os filtros deverão seguir preferencialmente direção única, das dimensões para a tabela fato.

Regras de Negócio

- Cada ocorrência deverá possuir um identificador único.
- Toda ocorrência deverá estar vinculada a uma data, loja, produto, categoria, supervisor e gerente válidos.
- A quantidade deverá ser maior que zero.
- O valor da perda deverá ser maior ou igual a zero.
- O tipo de perda deverá ser informado.



- Registros históricos não deverão ser excluídos fisicamente durante o processo normal de carga.
- A carga da tabela fato somente deverá ocorrer após a atualização das dimensões.
- Registros órfãos ou duplicados deverão ser rejeitados pelo processo ETL.
- As chaves organizacionais deverão representar os responsáveis no momento da ocorrência.

Implementação Física

A fPerdas foi projetada para suportar o maior volume de registros do Data Warehouse, mantendo integridade referencial, desempenho e escalabilidade.

A estrutura utiliza uma chave primária BIGINT, chaves estrangeiras inteiras e restrições de integridade para impedir registros inválidos.

Script SQL

```
CREATE TABLE fPerdas
(
    PerdaID BIGINT IDENTITY(1,1) NOT NULL,
    DataID INT NOT NULL,
    LojaID INT NOT NULL,
    ProdutoID INT NOT NULL,
    CategoriaID INT NOT NULL,
    SupervisorID INT NOT NULL,
    GerenteID INT NOT NULL,
    Quantidade INT NOT NULL,
    ValorPerda DECIMAL(18,2) NOT NULL,
    TipoPerda VARCHAR(50) NOT NULL,

    CONSTRAINT PK_fPerdas
        PRIMARY KEY CLUSTERED (PerdaID),

    CONSTRAINT FK_fPerdas_Calendarario
        FOREIGN KEY (DataID)
```



```

REFERENCES dCalendario(DataID),

CONSTRAINT FK_fPerdas_Loja
    FOREIGN KEY (LojaID)
    REFERENCES dLoja(LojaID),

CONSTRAINT FK_fPerdas_Produto
    FOREIGN KEY (ProdutoID)
    REFERENCES dProduto(ProdutoID),

CONSTRAINT FK_fPerdas_Categoria
    FOREIGN KEY (CategorialID)
    REFERENCES dCategoria(CategorialID),

CONSTRAINT FK_fPerdas_Supervisor
    FOREIGN KEY (SupervisorID)
    REFERENCES dSupervisor(SupervisorID),

CONSTRAINT FK_fPerdas_Gerente
    FOREIGN KEY (GerenteID)
    REFERENCES dGerente(GerenteID),

CONSTRAINT CK_fPerdas_Quantidade
    CHECK (Quantidade > 0),

CONSTRAINT CK_fPerdas_ValorPerda
    CHECK (ValorPerda >= 0),

CONSTRAINT CK_fPerdas_TipoPerda
    CHECK (LEN(LTRIM(RTRIM(TipoPerda))) > 0)
);

```



Constraints

Constraint	Campo/Relacionamento	Finalidade
PK_fPerdas	PerdaID	Identificar unicamente cada ocorrência.
FK_fPerdas_Calendario	DataID → dCalendario	Garantir data válida.
FK_fPerdas_Loja	LojaID → dLoja	Garantir loja válida.
FK_fPerdas_Produto	ProdutoID → dProduto	Garantir produto válido.
FK_fPerdas_Categoria	CategoriaID → dCategoria	Garantir categoria válida.
FK_fPerdas_Supervisor	SupervisorID → dSupervisor	Garantir supervisor válido.
FK_fPerdas_Gerente	GerenteID → dGerente	Garantir gerente válido.
CK_fPerdas_Quantidade	Quantidade > 0	Impedir quantidade nula ou negativa.
CK_fPerdas_ValorPerda	ValorPerda >= 0	Impedir valor financeiro negativo.
CK_fPerdas_TipoPerda	TipoPerda não vazio	Impedir classificação em branco.

Índices Recomendados

Como a tabela fato concentrará o maior volume de registros da plataforma, recomenda-se a criação dos seguintes índices:

```
CREATE INDEX IX_fPerdas_DataID  
ON fPerdas (DataID);
```

```
CREATE INDEX IX_fPerdas_LojaID  
ON fPerdas (LojaID);
```



```
CREATE INDEX IX_fPerdas_ProdutoID  
ON fPerdas (ProdutoID);
```

```
CREATE INDEX IX_fPerdas_CategoriaID  
ON fPerdas (CategoriaID);
```

```
CREATE INDEX IX_fPerdas_SupervisorID  
ON fPerdas (SupervisorID);
```

```
CREATE INDEX IX_fPerdas_GerenteID  
ON fPerdas (GerenteID);
```

```
CREATE INDEX IX_fPerdas_LojaID_DataID  
ON fPerdas (LojaID, DataID)  
INCLUDE  
(  
    ProdutoID,  
    CategoriaID,  
    Quantidade,  
    ValorPerda,  
    TipoPerda  
);
```

```
CREATE INDEX IX_fPerdas_ProdutoID_DataID  
ON fPerdas (ProdutoID, DataID)  
INCLUDE  
(  
    LojaID,  
    Quantidade,  
    ValorPerda  
);
```

Esses índices aceleram filtros por período, loja, produto, categoria, gerente e supervisor, além de otimizar consultas recorrentes do Power BI.



Exemplos de Registros

PerdaID	DataID	LojaID	ProdutoID	CategorialID	SupervisorID	GerenteID	Quantidade	ValorPerda	TipoPerda
1	20260115	10	1001	5	3	2	1	R\$ 3.499,90	Furto
2	20260116	12	2045	8	4	2	3	R\$ 149,70	Avaria
3	20260118	8	3100	6	2	1	2	R\$ 3.798,00	Divergência de Estoque
4	20260120	15	1280	4	5	3	4	R\$ 1.599,60	Quebra

Os registros apresentados são fictícios e possuem finalidade exclusivamente ilustrativa.

Consultas SQL de Exemplo

Valor total das perdas

```
SELECT
    SUM(ValorPerda) AS ValorTotalPerdas
FROM fPerdas;
```

Quantidade total de ocorrências

```
SELECT
    COUNT_BIG(PerdaID) AS TotalOcorrencias
FROM fPerdas;
```

Perdas por loja

```
SELECT
    l.CodigoLoja,
    l.NomeLoja,
    COUNT_BIG(f.PerdaID) AS TotalOcorrencias,
    SUM(f.Quantidade) AS QuantidadePerdida,
    SUM(f.ValorPerda) AS ValorTotalPerdido
FROM fPerdas f
INNER JOIN dLoja l
    ON f.LojaID = l.LojaID
GROUP BY
    l.CodigoLoja,
    l.NomeLoja
```

ORDER BY

ValorTotalPerdido DESC;

Perdas por categoria

SELECT

c.NomeCategoria,

COUNT_BIG(f.PerdaID) AS TotalOcorrencias,

SUM(f.Quantidade) AS QuantidadePerdida,

SUM(f.ValorPerda) AS ValorTotalPerdido

FROM fPerdas f

INNER JOIN dCategoria c

ON f.CategoriaID = c.CategoriaID

GROUP BY

c.NomeCategoria

ORDER BY

ValorTotalPerdido DESC;

Evolução mensal das perdas

SELECT

c.Ano,

c.MesNumero,

c.MesNome,

COUNT_BIG(f.PerdaID) AS TotalOcorrencias,

SUM(f.ValorPerda) AS ValorTotalPerdido

FROM fPerdas f

INNER JOIN dCalendario c

ON f.DataID = c.DataID

GROUP BY

c.Ano,

c.MesNumero,

c.MesNome

ORDER BY

c.Ano,

c.MesNumero;



Estratégia de Carga

A carga da fPerdas deverá ocorrer após a atualização das dimensões e seguirá as etapas abaixo:

- Extração das ocorrências do sistema de auditoria.
- Validação dos campos obrigatórios.
- Padronização dos tipos e formatos.
- Localização das chaves substitutas nas dimensões.
- Validação e rejeição de registros duplicados.
- Inserção dos registros válidos na tabela fato.
- Registro dos erros e inconsistências em log.
- Conciliação dos totais da origem com os totais carregados.

Validações do Processo ETL

Validação	Tratamento
Data inexistente na dCalendario	Rejeitar o registro e registrar erro.
Loja não localizada	Rejeitar o registro e registrar erro.
Produto não localizado	Rejeitar o registro e registrar erro.
Categoria não localizada	Rejeitar o registro e registrar erro.
Supervisor não localizado	Rejeitar o registro e registrar erro.
Gerente não localizado	Rejeitar o registro e registrar erro.
Quantidade menor ou igual a zero	Rejeitar o registro.
Valor negativo	Rejeitar o registro.
Tipo de perda vazio	Rejeitar ou classificar conforme regra aprovada.
Registro duplicado	Não inserir e registrar a ocorrência.



Utilização no Power BI

- Indicadores de valor total perdido.
- Quantidade total perdida.
- Número de ocorrências.
- Ranking de lojas, produtos e categorias.
- Análises por estado, gerente e supervisor.
- Evolução temporal e comparação entre períodos.
- Pareto de perdas.
- Análise por tipo de perda.
- Identificação de unidades críticas.

Indicadores Principais

Valor Total das Perdas

Valor Total das Perdas =
SUM (fPerdas[ValorPerda])

Quantidade Total Perdida

Quantidade Total Perdida =
SUM (fPerdas[Quantidade])

Total de Ocorrências

Total de Ocorrências =
COUNTROWS (fPerdas)

Valor Médio por Ocorrência

Valor Médio por Ocorrência =
DIVIDE (
[Valor Total das Perdas],
[Total de Ocorrências],
0
)

Performance

- Utilizar chaves inteiras e índices nas chaves estrangeiras.
- Aplicar carga incremental.



- Atualizar estatísticas periodicamente.
- Monitorar a fragmentação dos índices.
- Evitar consultas com SELECT *.
- Manter relacionamentos unidirecionais no Power BI.
- Avaliar particionamento, compressão ou Columnstore Index quando o volume alcançar dezenas de milhões de registros.

Possíveis Evoluções

- Número da auditoria.
- Documento de origem.
- Valor unitário e valor de custo.
- Valor recuperado.
- Status da ocorrência.
- Motivo detalhado.
- Responsável pelo registro.
- Data e hora da carga.
- Identificador do lote ETL.
- Indicador de reincidência.
- Ação corretiva e prazo de regularização.

Riscos e Cuidados Operacionais

- Duplicidade de ocorrências.
- Chaves dimensionais incorretas.
- Divergência entre a categoria do produto e a categoria armazenada na fato.
- Gerente ou supervisor incorreto para a data do evento.
- Valores financeiros inconsistentes.
- Exclusão de registros históricos.
- Reprocessamento indevido de períodos já carregados.
- Falha de sincronização entre dimensões e tabela fato.



Considerações Técnicas

A fPerdas representa o componente central da Audit Analytics Platform e concentra os eventos necessários para a construção dos indicadores de auditoria.

A implementação proposta combina integridade referencial, desempenho, rastreabilidade e escalabilidade, garantindo uma base consistente para os dashboards do Microsoft Power BI e para as análises estratégicas da Nexus Retail.

Objetos Auxiliares do Banco de Dados

Objetivo

Este capítulo apresenta os objetos auxiliares desenvolvidos para apoiar a utilização, manutenção e integração do banco de dados de plataforma **Audit Analytic Platform**.

Além das tabelas dimensionais e da tabela fato, a solução utiliza objetos complementares para simplificar consultas recorrentes, centralizar regras técnicas, padronizar processos de carga e disponibilizar dados consolidados para consumo pelo Microsoft Power BI.

Os objetos auxiliares contribuem para:

- Reduzir a complexidade das consultas realizadas pelos usuários;
- Padronizar o acesso aos dados;
- Evitar repetição de lógica SQL;
- Melhorar a legibilidade e manutenção dos scripts;
- Controlar processos de inclusão e atualização;
- Facilitar a integração com ferramentas analíticas;
- Aumentar a segurança ao limitar o acesso direto às tabelas;
- Melhorar a rastreabilidade das operações.

Na primeira versão da plataforma, serão utilizados principalmente:

- Views analíticas;
- Stored Procedures;
- Constraints;



- Defaults;
- Objetos de apoio ao processo de carga.

Padrões de Nomenclatura

Os objetos auxiliares deverão seguir as convenções estabelecidas no capítulo de desenvolvimento.

Objeto	Prefixo	Exemplo
View	vw_	vw_PerdasPorLoja
Stored Procedure	sp_	sp_CarregarPerdas
Constraint de validação	CK_	CK_fPerdas_ValorPerda
Default	DF_	DF_dProduto_Ativo
Chave primária	PK_	PK_fPerdas
Chave estrangeira	FK_	FK_fPerdas_Loja
Índice	IX_	IX_fPerdas_LojaID_DataID

Todos os nomes deverão utilizar uma estrutura clara, objetiva e compatível com o padrão PascalCase adotado para tabelas e campos.

Views Analíticas

Conceito

Views são objetos virtuais construídos a partir de consultas SQL armazenadas no banco de dados.

Eles não armazenam os registros de forma independente, mas apresentam os dados provenientes das tabela de maneira organizadas e adequada para determinado uso.

Na Audit Analytics Platform, as views serão utilizadas para:

- Consolidar dados de diferentes tabelas;
- Simplificar consultas frequentes;
- Disponibilizar informações preparadas para o Power BI;



- Padronizar os cálculos básicos;
- Reduzir a necessidade de usuários conhecerem toda a estrutura física;
- Aplicar uma camada adicional de segurança;
- Facilitar a manutenção das consultas.

Regras para Criação das Views

As views deverão seguir estas regras:

- Possuir prefixo vw_;
- Utilizar nomes que indiquem claramente sua finalidade;
- Retornar apenas as colunas necessárias;
- Evitar o uso de SELECT*;
- Utilizar aliases claros;
- Não aplicar ordenação interna sem necessidade técnica;
- Não incluir cálculos que devam ser centralizados no Power BI;
- Evitar lógica excessivamente complexa;
- Ser documentadas no controle de versões;
- Ser testadas antes da disponibilização.

Views Consolidada de Perdas

Objetivo

A view **vw_PerdasDetalhadas** tem como objetivo reunir, em uma única estrutura, os dados da tabela fato e os principais atributos das dimensões relacionadas.

Ela será utilizada como fonte para consultas operacionais, validações, análises exploratórias e, quando aplicável, integração com o Power BI.

Estrutura Disponibilizada

A view apresentará:

- Identificador da perda;
- Data da ocorrência;
- Ano, mês, trimestre e semestre;



- Código e nome da loja;
- Cidade, estado e região;
- Nome do gerente;
- Nome do supervisor;
- Código e nome do produto;
- Categoria;
- Quantidade;
- Valor da perda;
- Tipo da perda.

Script SQL

```
CREATE OR ALTER VIEW vw_PerdasDetalhadas
```

```
AS
```

```
SELECT
```

```
    f.PerdaID,
```

```
    f.DataID,
```

```
    c.DataCompleta,
```

```
    c.Dia,
```

```
    c.DiaSemanaNome,
```

```
    c.SemanaAno,
```

```
    c.MesNumero,
```

```
    c.MesNome,
```

```
    c.Trimestre,
```

```
    c.Semestre,
```

```
    c.Ano,
```

```
    f.LojaID,
```

```
    l.CodigoLoja,
```

```
    l.NomeLoja,
```

```
    l.Cidade,
```

```
    e.UF,
```

```
    e.Estado,
```

```
    e.Regiao,
```



```

f.GerenteID,

g.NomeGerente,

f.SupervisorID,

s.NomeSupervisor,

f.ProdutoID,

p.CodigoProduto,

p.NomeProduto,

f.CategoriaID,

cat.NomeCategoria,

f.Quantidade,

f.ValorPerda,

f.TipoPerda
FROM fPerdas AS f
INNER JOIN dCalendario AS c
    ON f.DataID = c.DataID
INNER JOIN dLoja AS l
    ON f.LojaID = l.LojaID
INNER JOIN dEstado AS e
    ON l.EstadoID = e.EstadoID
INNER JOIN dGerente AS g
    ON f.GerenteID = g.GerenteID
INNER JOIN dSupervisor AS s
    ON f.SupervisorID = s.SupervisorID
INNER JOIN dProduto AS p
    ON f.ProdutoID = p.ProdutoID
INNER JOIN dCategoria AS cat
    ON f.CategoriaID = cat.CategoriaID;

```

Justificativa Técnica

A criação dessa view reduz a repetição de operações JOIN em consultas recorrentes e fornece uma estrutura única para análise detalhadas.



Ela também facilita a validação do modelo, pois permite comparar, em uma mesma consulta, os identificadores da tabela fato e os atributos descritivos das dimensões.

A view substitui o modelo estrela no Power BI. Para a modelagem analítica principal, recomenda-se importar as dimensões de a tabela fato separadamente. A view poderá ser utilizada em relatórios auxiliares, verificações e cenários específicos.

Consulta de Exemplo

```
CREATE OR ALTER VIEW vw_PerdasPorLoja
AS
SELECT
    l.LojaID,
    l.CodigoLoja,
    l.NomeLoja,
    l.Cidade,
    e.UF,
    e.Estado,
    e.Regiao,

    COUNT_BIG(f.PerdaID) AS TotalOcorrencias,
    SUM(f.Quantidade) AS QuantidadePerdida,
    SUM(f.ValorPerda) AS ValorTotalPerdido,
    CAST(
        AVG(CAST(f.ValorPerda AS DECIMAL(18,2)))
        AS DECIMAL(18,2)
    ) AS ValorMedioPorOcorrencia
FROM fPerdas AS f
INNER JOIN dLoja AS l
    ON f.LojaID = l.LojaID
INNER JOIN dEstado AS e
    ON l.EstadoID = e.EstadoID
GROUP BY
    l.LojaID,
    l.CodigoLoja,
    l.NomeLoja,
```



l.Cidade,

e.UF,

e.Estado,

e.Regiao;

Utilização

A view poderá ser utilizada para:

- Ranking de lojas;
- Identificação de unidades críticas;
- Comparação entre estados;
- Relatórios gerenciais;
- Validação dos totais apresentados no Power BI;
- Exportação pontuais para Excel.

Views de Perdas por Produto

Objetivo

A view **vw_PerdasPorProduto** consolida as perdas por produto e categoria, permitindo identificar os itens com maior frequência de ocorrências e maior impacto financeiro.

Script SQL

```
CREATE OR ALTER VIEW vw_PerdasPorProduto
```

```
AS
```

```
SELECT
```

```
p.ProdutoID,
```

```
p.CodigoProduto,
```

```
p.NomeProduto,
```

```
cat.CategoriaID,
```

```
cat.NomeCategoria,
```

```
COUNT_BIG(f.PerdaID) AS TotalOcorrencias,
```

```
SUM(f.Quantidade) AS QuantidadePerdida,
```

```
SUM(f.ValorPerda) AS ValorTotalPerdido,
```

```
CAST(
```

```
AVG(CAST(f.ValorPerda AS DECIMAL(18,2)))
```




```

        AS DECIMAL(18,2)

    ) AS ValorMedioPorOcorrencia

FROM fPerdas AS f

INNER JOIN dProduto AS p

    ON f.ProdutoID = p.ProdutoID

INNER JOIN dCategoria AS cat

    ON f.CategoriaID = cat.CategoriaID

GROUP BY

    p.ProdutoID,

    p.CodigoProduto,

    p.NomeProduto,

    cat.CategoriaID,

    cat.NomeCategoria;

```

Views de Perdas Mensais

Objetivo

A view **vwPerdasMensais** consolida os resultados por ano e mês, facilitando análises de tendência e comparações temporais.

Script SQL

```

CREATE OR ALTER VIEW vw_PerdasMensais

AS

SELECT

    c.Ano,

    c.MesNumero,

    c.MesNome,

    c.AnoMes,

    COUNT_BIG(f.PerdalID) AS TotalOcorrencias,

    SUM(f.Quantidade) AS QuantidadePerdida,

    SUM(f.ValorPerda) AS ValorTotalPerdido,

    CAST(

        AVG(CAST(f.ValorPerda AS DECIMAL(18,2)))

        AS DECIMAL(18,2)

    ) AS ValorMedioPorOcorrencia

```



```
FROM fPerdas AS f

INNER JOIN dCalendario AS c

    ON f.DataID = c.DataID

GROUP BY

    c.Ano,

    c.MesNumero,

    c.MesNome,

    c.AnoMes;
```

Views de Perdas por Responsável

Objetivo

A view **vw_PerdasPorResponsavel** consolida os indicadores por gerente e supervisor, apoiando a avaliação da estrutura organizacional.

Script SQL

```
CREATE OR ALTER VIEW vw_PerdasPorResponsavel

AS

SELECT

    g.GerenteID,

    g.NomeGerente,

    s.SupervisorID,

    s.NomeSupervisor,

    COUNT_BIG(f.PerdaID) AS TotalOcorrencias,

    SUM(f.Quantidade) AS QuantidadePerdida,

    SUM(f.ValorPerda) AS ValorTotalPerdido

FROM fPerdas AS f

INNER JOIN dGerente AS g

    ON f.GerenteID = g.GerenteID

INNER JOIN dSupervisor AS s

    ON f.SupervisorID = s.SupervisorID

GROUP BY

    g.GerenteID,

    g.NomeGerente,

    s.SupervisorID,

    s.NomeSupervisor;
```



Segurança das Views

O acesso aos dados poderá ser concedido por meio das views, evitando que determinados usuários consultem diretamente as tabelas físicas.

Exemplo:

```
GRANT SELECT  
ON vw_PerdasDetalhadas  
TO DesenvolvedorBI;
```

```
GRANT SELECT  
ON vw_PerdasPorLoja  
TO AnalistaAuditoria;
```

```
GRANT SELECT  
ON vw_PerdasMensais  
TO AnalistaAuditoria;
```

Essa abordagem permite controlar quais conjuntos de dados estarão disponíveis para cada perfil, reduzindo a exposição desnecessária das estruturas internas.

Considerações Técnicas sobre Views

As views deverão ser utilizadas para simplificar o consumo e padronizar consultas, mas não devem suplicar toda a lógica analítica do Power BI.

Os cálculos dependentes do contexto de filtros, como participação percentual, acumulados, comparação anual e indicadores dinâmicos, deverão permanecer preferencialmente em medidas DAX.

As views deverão concentrar:

- Junções recorrentes;
- Colunas descritivas;
- Consolidação estáveis;
- Regras técnicas reutilizáveis;
- Estruturas necessárias para validação e integração.



Stored Procedures

Objetivo

As **Stored Procedures** constituem rotinas armazenadas no Microsoft SQL Server responsável por automatizar operações recorrentes da Audit Analytics Platform.

Sua utilização permite centralizar regras de negócio, reduzir a repetição de código, aumentar a segurança das operações e facilitar a manutenção do ambiente analítico.

Na plataforma, as procedures serão empregadas principalmente para:

- Automatizar cargas do Data Warehouse;
- Atualizar dimensões;
- Inserir registros na tabela fato;
- Validar consistência dos dados;
- Registros logs de processamento;
- Executar rotinas administrativas.

A utilização de procedures reduz a complexidade das aplicações clientes e garante que as regras críticas permaneçam centralizadas no banco de dados.

Convenção de Nomenclatura

Todas as procedures deverão utilizar o prefixo **sp_**, seguindo de um nome que represente claramente sua finalidade.

Procedure	Finalidade
sp_CarregarCalendario	Gerar registros da dimensão calendário
sp_CarregarProduto	Atualizar a dimensão de produtos
sp_CarregarPerdas	Inserir registros na tabela fato
sp_ReprocessarCarga	Reexecutar uma carga específica
sp_AtualizarDimensoes	Atualizar todas as dimensões
sp_ValidarIntegridade	Executar validações gerais



Procedure de Carga da Dimensão Calendário

Objetivo

Gera automaticamente todos os registros da dimensão **dCalendario** dentro do intervalo informado.

Essa procedure deverá ser executada:

- Na implantação inicial;
- Sempre que for necessário ampliar o calendário.

Parâmetros

Parâmetro	Tipo	Descrição
@DataInicial	DATE	Primeira data a ser criada
@DataFinal	DATE	Última data a ser criada

Exemplo de execução

EXEC sp_CarregarCalendario

@DataInicial = '2020-01-01';

@DataFinal = '2035-12-31';

Procedure de Atualização da Dimensão Produto

Objetivo

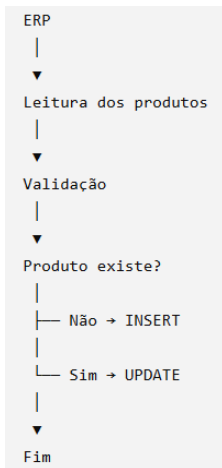
Atualizar o cadastro de produtos proveniente do sistema ERP.

A procedure deverá:

- Localizar produtos existentes;
- Inserir novos produtos;
- Atualizar informações alteradas;
- Manter produtos inativos para preservar o histórico.

Fluxo





Procedure de Carga da Tabela Fato

Objetivo

Inserir as ocorrências de perdas na tabela **fPerdas**.

A procedure deverá garantir que todos os registros possuam relacionamento válido com as dimensões.

Etapas executadas

1. Ler dados da origem.
2. Validar campos obrigatórios.
3. Localizar chaves das dimensões.
4. Verificar duplicidade.
5. Inserir registros.
6. Registrar log.
7. Finalizar carga.

Stored Procedures implementadas

Procedure	Finalidade
sp_CarregarCalendario	Carga incremental da dimensão calendário
sp_CarregarProduto	Inserção e atualização de produtos
sp_CarregarPerda	Registro de ocorrências de perdas



As Stored Procedures implementadas representam os principais processos operacionais do ambiente analítico. Novas procedures poderão ser adicionadas futuramente conforme a evolução da plataforma.

Procedure de Reprocessamento

Em situações excepcionais poderá ser necessário reprocessar determinado período.

A procedure de reprocessamento deverá:

- Excluir apenas registros autorizados;
- Executar novamente a carga;
- Registrar a operação em log;
- Impedir duplicidades.

Procedure de Validação

Antes da carga da tabela fato deverá ser executada uma rotina de validação.

Serão verificadas:

- Existência das dimensões;
- Integridade referencial;
- Valores negativos;
- Registros duplicados;
- Datas inválidas;
- Produtos inexistentes;
- Lojas inexistentes;
- Categorias inexistentes;
- Gerentes inexistentes;
- Supervisores inexistentes.

Caso alguma inconsistência seja identificada, a carga deverá ser interrompida e registrada em log.

Benefícios da Utilização de Procedures

A adoção de procedures proporciona diversas vantagens para a plataforma:



- Centralização das regras de negócio;
- Redução da duplicidade de código;
- Melhor desempenho em operações repetitivas;
- Maior segurança, restringindo acesso direto às tabelas;
- Facilidade de manutenção e evolução;
- Padronização dos processos de carga;
- Rastreabilidade das operações;
- Simplificação das aplicações clientes.

Considerações Técnicas

As Stored Procedures serão responsáveis pelas operações críticas da Audit Analytics Platform.

Sua utilização garante maior controle sobre os processos de ETL, reduz o risco de inconsistências e facilita futuras evoluções da solução.

Todas as procedures deverão ser versionadas em conjunto com os scripts do banco de dados, assegurando rastreabilidade e controle das alterações.

Estratégia Geral da Indexação

Objetivo

A estratégia de indexação tem como objetivo otimizar o desempenho das consultas realizadas na Audit Analytics Platform, reduzindo o tempo de resposta das operações de leitura e melhorando a eficiência dos dashboards desenvolvidos no Microsoft Power BI.

Considerando que a tabela **fPerdas** poderá atingir milhões de registros ao longo do tempo, a definição adequada de índices torna-se um fator crítico para garantir escalabilidade, desempenho e estabilidade da solução.

Além disso, uma política consistente de indexação contribui para a redução do consumo de recursos do servidor, melhora o tempo de execução das consultas analíticas e facilita o crescimento do Data Warehouse,



Conceitos de Indexação

Os índices funcionam como estruturas auxiliares que permitem ao SQL Server localizar registros de maneira mais eficiente, evitando a necessidade de percorrer toda a tabela para responder a uma consulta.

De forma simplificada, um índice pode ser comparado ao índice de um livro: em vez de percorrer todas as páginas até encontrar um determinado assunto, basta consultar o índice para localizar rapidamente a informação desejada.

Na Audit Analytics Platform, a utilização adequada de índices permitirá que consultas envolvendo milhões de registros sejam executadas em poucos segundos, preservando a experiência dos usuários e o desempenho dos relatórios.

Tipos de Índices Utilizados

Clustered Index

O índice clustered determina a organização física dos registros na tabela.

Cada tabela poderá possuir apenas um índice clustered, normalmente definido sobre a chave primária.

Na plataforma, todas as tabelas utilizarão índices clustered sobre suas respectivas chaves primárias.

Exemplos:

Tabela	Índice Clustered
dCalendario	DataID
dEstado	EstadoID
dLoja	LojaID
dSupervisor	SupervisorID
dGerente	GerenteID
dCategoria	CategoriaID
dProduto	ProdutoID
fPerdas	PerdaID



Nonclustered Index

Os índices nonclustered são utilizados para acelerar consultas específicas sem alterar a organização física da tabela.

Serão empregados principalmente em:

- Chaves estrangeiras;
- Colunas utilizadas em filtros;
- Campos frequentemente utilizados em agrupamentos;
- Colunas presentes em cláusulas JOIN.

Índices Compostos

Índices compostos armazenam mais de uma coluna e são recomendados para consultas que utilizam múltiplos filtros simultaneamente.

Exemplo:

LojaID + DataID

Esse índice melhora significativamente consultas como:

- Perdas por loja em determinado período;
- Evolução mensal por unidade;
- Relatórios operacionais.

Estratégia de Criação dos Índices

A criação dos índices seguirá as seguintes diretrizes:

- Priorizar colunas utilizadas em relacionamentos.
- Indexar chaves estrangeiras da tabela fato.
- Evitar índices redundantes.
- Avaliar custo de manutenção antes da criação.
- Criar índices compostos apenas quando houver ganho comprovado de desempenho.
- Revisar periodicamente os índices com base nas consultas executadas.



Índices Implementados

dCalendario

Índice	Colunas
PK_dCalendario	DataID
IX_dCalendario_DataCompleta	DataCompleta
IX_dCalendario_AnoMes	Ano, MesNumero

dEstado

Índice	Colunas
PK_dEstado	EstadoID
UQ_dEstado_UF	UF

dLoja

Índice	Colunas
PK_dLoja	LojaID
IX_dLoja_Codigo	CodigoLoja
IX_dLoja_Estado	EstadoID

dSupervisor

Índice	Colunas
PK_dSupervisor	SupervisorID

dGerente



Índice	Colunas
PK_dGerente	GerenteID

dCategoria

Índice	Colunas
PK_dCategoria	CategoriaID

dProduto

Índice	Colunas
PK_dProduto	ProdutoID
IX_dProduto_Codigo	CodigoProduto
IX_dProduto_Categoria	CategoriaID

fPerda

Índice	Colunas
PK_fPerdas	PerdaID
IX_fPerdas_Data	DataID
IX_fPerdas_Loja	LojaID
IX_fPerdas_Produto	ProdutoID
IX_fPerdas_Categoria	CategoriaID
IX_fPerdas_Supervisor	SupervisorID
IX_fPerdas_Gerente	GerenteID
IX_fPerdas_Loja_Data	LojaID, DataID
IX_fPerdas_Produto_Data	ProdutoID, DataID



Benefícios Esperados

A estratégia de indexação proporciona diversos benefícios para o ambiente analítico.

Entre os principais destacam-se:

- Redução do tempo de resposta das consultas.
- Maior velocidade na atualização dos dashboards.
- Melhor desempenho das operações de agrupamento.
- Otimização dos relacionamentos entre tabelas.
- Redução do consumo de CPU em consultas complexas.
- Menor tempo de processamento durante análises históricas.
- Escalabilidade para crescimento do volume de dados.

Manutenção dos Índices

Para preservar o desempenho do banco de dados, os índices deverão passar por rotinas periódicas de manutenção.

As principais atividades incluem:

- Atualização das estatísticas do banco de dados.
- Reorganização de índices com fragmentação moderada.
- Reconstrução de índices altamente fragmentados.
- Monitoramento do crescimento das tabelas.
- Avaliação de índices pouco utilizados.
- Remoção de índices redundantes.

Essas atividades deverão ser executadas em janelas de manutenção previamente definidas, minimizando impactos nas operações da plataforma.

Monitoramento de Performance

O monitoramento contínuo permitirá identificar oportunidades de otimização e antecipar problemas de desempenho.

Entre os indicadores monitorados destacam-se:



- Tempo médio das consultas.
- Utilização dos índices.
- Percentual de fragmentação.
- Crescimento da tabela fato.
- Consumo de CPU.
- Consumo de memória.
- Tempo de atualização do Power BI.
- Volume diário de registros processados.

Considerações Técnicas

A estratégia de indexação definida para a Audit Analytics Platform foi planejada para equilibrar desempenho de leitura, custo de manutenção e crescimento futuro do ambiente.

A combinação de índices clustered, nonclustered e compostos garante suporte eficiente às consultas analíticas mais frequentes, preservando a integridade do modelo dimensional e a escalabilidade da solução.

Adoção dessas práticas permitirá que a plataforma mantenha bom desempenho mesmo com o aumento significativo do volume de dados, assegurando tempos de resposta adequados para os processos de auditoria e para os dashboards executivos.

Estratégia de Carga de Integração (ETL)

Objetivo

A Estratégia de Carga e Integração define o processo responsável pela extração, transformação e carregamento (RTL – Extract, Transform and Load) dos dados utilizados pela **Audit Analytics Platform**.

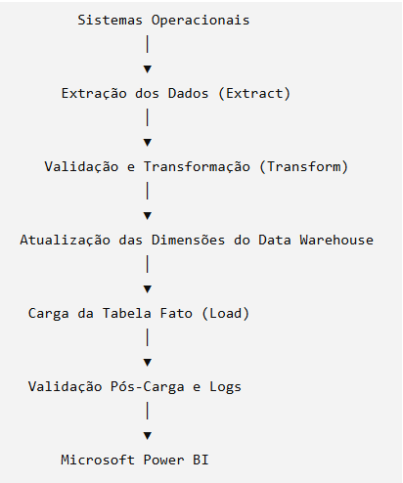
Seu objetivo é garantir que as informações provenientes dos sistemas operacionais sejam incorporadas ao Data Warehouse de forma íntegra, consistente e padronizada, preservando a qualidade dos dados utilizados nos processos analíticos e na geração de indicadores estratégicos.

A arquitetura de ETL foi projetada para suportar crescimento contínuo do volume de dados, permitindo futuras expansões sem comprometer o desempenho da plataforma.



Visão Geral do Processo ETL

O fluзо de ETL da plataforma segue uma sequência estruturada de etapas, garantindo que apenas dados válidos sejam disponibilizados para análise.



Esse fluxo assegura que as dimensões estejam atualizadas antes da carga da tabela fato, evitando inconsistências de relacionamento.

Fonte de Dados

A plataforma poderá integrar dados provenientes de diferentes sistemas corporativos.

Fonte	Finalidade
Sistema ERP	Cadastro de produtos e lojas
Sistema de Auditoria	Ocorrências de perdas
Planilhas de apoio	Informações complementares
Banco Operacional	Dados históricos
APIs corporativas	Integrações futuras

Todas as fontes deverão passar pelo processo de validação antes da carga no Data Warehouse.

Etapa de Extração (Extract)

A etapa de extração consiste na obtenção dos dados das fontes operacionais.

Durante essa fase serão realizadas:

- Conexão com as bases de origem;
- Leitura dos registros;
- Identificação de novos dados;
- Captura de alterações;
- Registros da data e hora da extração;
- Geração de logs de execução.

A extração deverá ser realizada de forma incremental sempre que possível, reduzindo o tempo de processamento.

Etapas de Transformação (Transform)

Após a extração, os dados passam por um conjunto de transformações para adequação ao modelo dimensional.

As principais transformações incluem:

- Conversão de tipos de dados;
- Padronização de textos;
- Remoção de espaços em branco;
- Conversão de datas;
- Tratamento de valores nulos;
- Eliminação de registros suplicados;
- Padronização de códigos;
- Validação de integridade referencial;
- Identificação de inconsistências.

Essa etapa é essencial para garantir a confiabilidade das análises.

Atualização das Dimensões

Ante de carga da tabela fato, todas as dimensões deverão ser atualizadas.

A sequência recomendada é:

1. dCalendario
2. dEstado



3. dSupervisor
4. dGerente
5. dCategoria
6. dLoja
7. dProduto

Essa ordem respeita as dependências entre as tabelas e evita violações de integridade referencial.

Carga da Tabela Fato

Somente após a atualização das dimensões será iniciada a carga da tabela **fPerdas**.

Durante essa etapa serão executadas as seguintes validações:

- Existência da data na dCalendario;
- Existência da loja na dLoja;
- Existência do produto na dProduto;
- Existência da categoria na dCategoria;
- Existência do supervisor na dSupervisor;
- Existência do gerente na dGerente;
- Quantidade maior que zero;
- Valor da perda maior ou igual a zero;
- Tipo de perda informado.

Registros que não atenderem às regras deverão ser rejeitados e registrado em log.

Estratégia de Carga Incremental

Para produzir o tempo de processamento e evitar recargas desnecessárias, será adotada uma estratégia de carga incremental.

O processo consiste em identificar apenas os registros novos ou alterados desde a última execução do ETL.

Benefícios

- Redução do tempo de processamento;



- Menor consumo de recursos do servidor;
- Menor impacto sobre o sistema de origem;
- Atualização mais frequente dos dados;
- Escalabilidade da solução.

Tratamento de Erros

Durante o processo ETL poderão ocorrer diferentes tipos de inconsistências.

Entre as principais estão:

Erro	Tratamento
Produto inexistente	Registrar log e rejeitar registro
Loja inexistente	Registrar log e rejeitar registro
Categoria inexistente	Registrar log e rejeitar registro
Data inválida	Registrar log e rejeitar registro
Valor negativo	Registrar log e rejeitar registro
Quantidade igual a zero	Registrar log e rejeitar registro
Registro duplicado	Ignorar ou registrar conforme política definida

Nenhum registro inconsistente deverá ser carregado na tabela fato.

Logs de Processamento

Cada execução do ETL deverá gerar um registro contendo informações sobre a operação realizada.

Os logs deverão armazenar:

- Data e hora de início;
- Data e hora de término;
- Usuário ou processo responsável;
- Quantidade de registros lidos;
- Quantidade de registros inseridos;



- Quantidade de registros atualizados;
- Quantidade de registro rejeitados;
- Motivos das rejeições;
- Tempo total de processamento;
- Status da execução.

Essas informações são essenciais para auditoria e rastreabilidade.

Frequência das Cargas

A periodicidade de atualização dos dados deverá ser definida conforme a necessidade operacional da empresa.

Para a primeira versão da plataforma, recomenda-se

Processo	Frequência
Atualização das dimensões	Diária
Carga da tabela fato	Diária
Atualização do Power BI	Após conclusão do ETL
Reconstrução de índices	Semanal
Atualização de estatísticas	Semanal

Em cenários futuros, a frequência poderá ser ampliada para múltiplas execuções diárias.

Segurança do Processo ETL

O processo de ETL deverá ser executado por contas de serviço específicas, com permissões restritas às operações necessárias.

Boas práticas recomendadas:

- Utilização de usuários dedicados para carga;
- Armazenamento seguro de credenciais;
- Registro de todas as execuções;
- Restrição de acesso às rotinas de carga;



- Validação das permissões antes da execução.

Boas Práticas

Para garantir a qualidade do processo ETL, deverão ser observadas as seguintes práticas:

- Executar as cargas em ambiente controlado;
- Validar previamente os dados de origem;
- Documentar alterações nas regras de transformação;
- Monitorar o tempo de execução das cargas;
- Revisar periodicamente os logs;
- Manter histórico das execuções;
- Automatizar o processo sempre que possível;
- Testar novas versões em ambiente de homologação antes da produção.

Considerações Técnicas

A estratégia de ETL definida para a Audit Analytics Platform assegura que os dados sejam incorporados ao Data Warehouse de maneira padronizada, íntegra e rastreável.

A adoção de cargas incrementais, validações de integridade e geração de logs contribui para a confiabilidade do ambiente analítico e para a qualidade dos indicadores disponibilizados aos usuários.

Além disso, a arquitetura proposta oferece flexibilidade para futuras integrações com novas fontes de dados e aumento do volume de informações, preservando a escalabilidade e a eficiência operacional da solução.

Controle de Transações e Tratamento de Erros

Objetivo

Este capítulo apresenta a estratégia adotada para o controle de transações e tratamento de erros na **Audit Analytics Platform**.

O objetivo é garantir que todas as operações realizadas no banco de dados ocorram de forma segura, preservando a integridade das informações mesmo em situação de falha durante os processos de carga, atualização ou manutenção.



A implementação dessas práticas reduz o risco de inconsistências, evita a gravação parcial de dados e assegura que o ambiente permaneça confiável para análises operacionais e estratégicas.

Conceito de Transação

Uma transação representa um conjunto de operações executadas como uma única unidade lógica de trabalho.

Todas as instruções pertencentes à mesma transação deverão ser concluídas com sucesso para que as alterações sejam efetivamente gravadas no banco de dados.

Caso qualquer etapa apresente erro, todas as alterações realizadas durante a transação deverão ser desfeitas (rollback), retornando o banco ao estado anterior.

Esse comportamento garante que não existam registros parcialmente processados.

Propriedades ACID

As transações implementadas deverão respeitar as propriedades ACID, amplamente utilizadas em bancos de dados relacionais.

Atomicidade (Atomicity)

Todas as operações pertencentes à mesma transação serão executadas integralmente ou totalmente canceladas.

Exemplo:

Durante carga da tabela **fPerdas**, se ocorrer erro ao inserir um registro, toda a carga da transação será revertida.

Consistência (Consistency)

Cada transação deverá preservar todas as regras de integridade definidas para o banco de dados.

Exemplos:

- Não permitir chaves estrangeiras inválidas.
- Não permitir valores negativos,
- Respeitar todas as constraints.
- Manter a integridade referencial entre dimensões e tabela fato.



Isolamento (Isolation)

Transações executadas simultaneamente não deverão interferir entre si.

Cada processo deverá visualizar apenas dados consistentes.

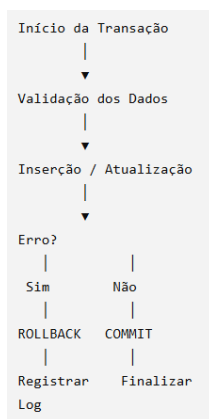
Durabilidade (Durability)

Após a confirmação (COMMIT), os dados permanecerão gravados mesmo em caso de falha do sistema ou reinicialização do servidor.

Controle de Transações

Os processos críticos deverão utilizar controle explícito de transações.

Fluxo recomendado.



Essa abordagem garante que alterações parciais não sejam persistidas.

Estrutura Recomendada

As procedures responsáveis pelas cargas deverão utilizar a seguinte estrutura:

```
BEGIN TRY

    BEGIN TRANSACTION;

    -- Operações de carga

    COMMIT TRANSACTION;

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION;

    -- Registro do erro

END CATCH;
```

Essa estrutura reduz significativamente o risco de inconsistências durante o processo ETL.



Tratamento de Erros

Todos os erros identificados durante os processos de carga deverão ser tratados de forma controlada.

As principais situações previstas são:

Situação	Ação
Produto inexistente	Cancelar inserção e registrar log
Loja inexistente	Cancelar inserção e registrar log
Categoria inexistente	Cancelar inserção e registrar log
Supervisor inexistente	Cancelar inserção e registrar log
Gerente inexistente	Cancelar inserção e registrar log
Data inexistente	Cancelar inserção e registrar log
Quantidade inválida	Rejeitar registro
Valor negativo	Rejeitar registro
Registro duplicado	Ignorar ou registrar ocorrência
Violação de chave estrangeira	Executar rollback

Registro de Logs

Todo erro deverá ser registrado para posterior análise.

Os logs deverão conter:

Informação	Descrição
Data/Hora	Momento da ocorrência
Processo	Procedure executada
Usuário	Responsável pela execução
Tipo do erro	Classificação
Mensagem	Descrição detalhada



Informação	Descrição
Linha SQL	Local da falha
Severidade	Nível do erro
Status	Tratado ou pendente

Essas informações permitem rastrear falhas e facilitar sua correção.

Exemplo de Estrutura de Log

Data	Procedure	Erro	Mensagem
25/07/2026 14:10	sp_CarregarPerdas	FK	Produto inexistente
25/07/2026 14:12	sp_CarregarPerdas	CHECK	Valor negativo
25/07/2026 14:15	sp_CarregarProduto	PK	Produto duplicado

Estratégia de Recuperação

Após a ocorrência de falhas, o processo deverá seguir as etapas abaixo:

1. Identificar o erro.
2. Cancelar a transação.
3. Registrar o evento em log.
4. Notificar o responsável.
5. Corrigir a inconsistência.
6. Reexecutar a carga.
7. Validar os resultados obtidos.

Essa estratégia reduz a possibilidade de perda de dados e facilita a retomada do processamento.

Boas Práticas

Para garantir maior confiabilidade do ambiente, recomenda-se:

- Utilizar transações apenas quando necessário;



- Evitar transações excessivamente longas;
- Realizar validações antes das inserções;
- Utilizar TRY...CATH em procedures críticas;
- Registrar todos os erros relevantes;
- Revisar periodicamente os logs;
- Monitorar falhas recorrentes;
- Documentar alterações nas regras de tratamento de erros.

Benefícios

A adoção de mecanismos de controle de transações proporciona diversos benefícios:

- Maior integridade dos dados;
- Redução de inconsistências;
- Facilidade de auditoria;
- Melhor rastreabilidade;
- Recuperação rápida após falhas;
- Maior confiabilidade do processo ETL;
- Segurança nas atualizações do banco de dados.

Exemplo de Estrutura de Log

A estratégia de controle de transações implementada na Audit Analytocs Platform busca assegurar que todas as operações críticas ocorram de forma segura e consistente.

A utilização de transações explícitas, tratamento estruturado de execuções e registro detalhado dos logs reduz significativamente o risco de inconsistências e aumenta a confiabilidade do ambiente analítico.

Essas práticas seguem recomendações consolidadas para administração de banco de dados relacionais e contribuem para a manutenção da integridade das informações ao longo de todo o ciclo de vida da solução.



Monitoramento e Manutenção

Objetivo

Este capítulo apresenta as diretrizes para o monitoramento contínuo e a manutenção preventiva do banco de dados da **Audit Analytics Platform**.

O objetivo é assegurar que o ambiente permaneça disponível, confiável e com desempenho adequado ao longo do tempo, mesmo diante do crescimento do volume de dados e do aumento da qualidade de usuários e consultas analíticas.

As atividades descritas neste capítulo visam identificar problemas de forma proativa, reduzir riscos operacionais e garantir a continuidade dos processos de análise de dados.

Estratégia de Monitoramento

O monitoramento deverá ser realizado de forma contínua, permitindo acompanhar o comportamento do banco de dados e identificar possíveis degradações de desempenho antes que impactem os usuários.

Entre os principais aspectos monitorados destacam-se:

- Disponibilidade do banco de dados;
- Utilização de CPU;
- Consumo de memória;
- Espaço em disco;
- Tempo médio de resposta das consultas;
- Crescimento das tabelas;
- Utilização dos índices;
- Fragmentação dos índices;
- Tempo de execução das rotinas ETL.
- Quantidade de registros processados.

O acompanhamento desses indicadores permitirá identificar tendências e planejar ações preventivas.



Indicadores de Desempenho

Os principais indicadores (KPIs) para monitoramento do ambiente são apresentados a seguir:

Indicador	Objetivo
Tempo médio das consultas	Avaliar desempenho das operações
Tempo do processo ETL	Medir eficiência das cargas
Utilização de CPU	Identificar sobrecarga do servidor
Consumo de memória	Verificar disponibilidade de recursos
Espaço em disco	Planejar expansão de armazenamento
Fragmentação dos índices	Definir necessidade de manutenção
Registros processados por carga	Avaliar crescimento da base
Ocorrências de erro	Monitorar estabilidade do ambiente

Esses indicadores deverão ser revisados periodicamente para garantir que o ambiente continue atendendo aos requisitos de desempenho.

Rotinas de Manutenção Preventiva

Para preservar a integridade e o desempenho do banco de dados, deverão ser executadas rotinas periódicas de manutenção preventiva.

As principais atividades incluem:

- Atualização das estatísticas do banco;
- Reorganização de índices;
- Reconstrução de índices fragmentados;
- Verificação da integridade das tabelas;
- Limpeza de registros temporários;
- Revisão dos logs de execução;
- Monitoramento do crescimento do banco de dados;



- Validação das permissões de acesso.

Essas ações contribuem para reduzir o risco de falhas e manter a eficiência das consultas.

Atualização das Estatísticas

As estatísticas do banco de dados fornecem informações utilizadas pelo otimizador de consultas SQL Server.

A atualização periódica dessas estatísticas é essencial para que o mecanismo de banco de dados escolha os planos de execução mais eficientes.

Recomenda-se que essa atividade seja realizada semanalmente ou sempre que houver alterações significativas no volume de dados.

Manutenção dos Índices

Os índices poderão sofrer fragmentação em decorrência das operações de inserção, atualização e exclusão de registros.

Para minimizar impactos no desempenho, recomenda-se:

- Reorganizar índices com fragmentação moderada;
- Reconstruir índices altamente fragmentados;
- Avaliar índices pouco utilizados;
- Remover índices redundantes quando necessário.

A manutenção deverá ser realizada preferencialmente em janelas programadas.

Monitoramento das Rotinas ETL

As rotinas de ETL deverão ser acompanhadas continuamente para garantir que os processos de carga sejam executados conforme planejado.

Os principais itens monitorados incluem:

- Horário de início e término das cargas;
- Tempo total de processamento;
- Quantidade de registros processados;
- Registros rejeitados;
- Erros ocorridos durante a execução;
- Consumo de recursos do servidor.



A análise desses dados permite identificar gargalos e oportunidades de otimização.

Monitoramento do Crescimento da Base

O crescimento do banco de dados deverá ser acompanhado regularmente para garantir que haja capacidade de armazenamento suficiente.

Entre os aspectos avaliados estão:

- Tamanho total do banco;
- Crescimento da tabela fPerdas;
- Crescimento das dimensões;
- Espaço livre disponível;
- Taxa média de crescimento mensal.

Essas informações auxiliam no planejamento de capacidade e evitam interrupções causadas por falta de espalho em disco.

Gestão de Logs

Os logs gerados pelo banco de dados e pelas rotinas ETL deverão ser armazenados de forma organizada e mantidos pelo período definido pela política da organização.

A gestão dos logs deverá contemplar:

- Registro de erros;
- Registro de cargas executadas;
- Histórico de atualizações;
- Registro de acessos administrativos;
- Limpeza periódica de logs obsoletos.

A manutenção adequada dos logs facilita auditorias e investigações de incidentes.

Segurança Operacional

Durante as atividades de manutenção deverão ser observadas práticas que garantam a segurança do ambiente.

Entre elas:

- Execução de rotinas por usuários autorizados;



- Registro das atividades administrativas;
- Aplicação de atualizações de segurança do SQL Server;
- Utilização de conexões seguras para acesso ao banco de dados.

Cronograma de Manutenção

A tabela a seguir apresenta uma sugestão de periodicidade para as principais atividades de manutenção.

Atividade	Periodicidade
Atualização das estatísticas	Semanal
Reorganização de índices	Semanal
Reconstrução de índices	Mensal ou conforme necessidade
Verificação de integridade	Mensal
Revisão dos logs	Semanal
Monitoramento do crescimento da base	Mensal
Revisão das permissões de acesso	Trimestral
Atualização do SQL Server	Conforme disponibilidade de versões e correções

Esse cronograma poderá ser ajustado conforme o volume de dados e as características do ambiente de produção.

Benefícios da Estratégia de Monitoramento

A adoção das práticas descritas neste capítulo proporciona diversos benefícios para a operação da Audit Analytics Platform, entre eles:

- Maior disponibilidade do banco de dados;
- Redução do risco de falhas operacionais;
- Melhor desempenho das consultas analíticas;
- Planejamento adequado de capacidade;



- Identificação precoce de problemas;
- Maior confiabilidade das rotinas ETL;
- Apoio às atividades de auditoria e governança.

Considerações Técnicas

O monitoramento contínuo e a manutenção preventiva são elementos fundamentais para garantir a estabilidade do ambiente de banco de dados da Audit Analytics Platform.

A implementação das rotinas descritas neste capítulo permite preservar o desempenho, reduzir indisponibilidades e assegurar que a plataforma continue atendendo às necessidades da organização mesmo com o crescimento do volume de dados e da complexidade das análises.

Backup e Recuperação

Objetivo

Este capítulo estabelece as diretrizes para a realização de backups e os procedimentos de recuperação do banco de dados da **Audit Analytics Platform**.

O objetivo é garantir a disponibilidade, integridade e continuidade das informações em situações de falhas de hardware, erro operacionais, corrupção de dados, ataque cibernéticos ou desastres que possam comprometer o ambiente de banco de dados.

A estratégia de backup descrita neste documento visa minimizar a perda de dados e reduzir o tempo necessário para restabelecer os serviços, assegurando a continuidade das operações analíticas da plataforma.

Política de Backup

A política de backup deverá contemplar diferentes tipos de cópias de segurança , de forma a equilibrar proteção, desempenho e utilização de recursos de armazenamento.

Os principais tipos de backup adotados serão:

Tipo de Backup	Finalidade
Completo (Full Backup)	Cópia integral do banco de dados



Tipo de Backup	Finalidade
Diferencial	Armazenar alterações desde o último backup completo
Log de Transações	Registrar todas as transações realizadas após o último backup de log

Essa combinação permite recuperar o ambiente com maior precisão e reduzir o tempo de restauração.

Estratégia de Backup

A estratégia proposta considera o volume de dados esperados e a necessidade de disponibilidade do ambiente.

Processo	Frequência Recomendada
Backup Completo	Semanal
Backup Diferencial	Diário
Backup do Log de Transações	A cada hora
Validação dos Backups	Semanal
Teste de Restauração	Trimestral

A frequência poderá ser ajustada conforme o crescimento da base de dados e os requisitos operacionais da organização.

Armazenamento dos Backups

Os arquivos de backup deverão ser armazenados em locais seguros e protegidos contra perda física ou lógica

Recomenda-se a utilização de uma estratégia baseada no princípio **3-2-1**:

- Manter **três cópias** dos dados;
- Utilizar **dois meios de armazenamento distintos**;
- Armazenar **uma cópia em local externo** (off-site ou em nuvem).

Essa abordagem reduz significativamente os riscos associados à perda de dados.



Procedimento de Recuperação

Em caso de falha, a recuperação do banco de ados deverá seguira as etapas abaixo:

- 1. Identificar a causa da falha.
- 2. Interromper novas gravações no banco de dados.
- 3. Restaurar o último backup completo.
- 4. Aplicar p backup diferencial mais recente.
- 5. Aplicar os logs de transações disponíveis.
- 6. Validar a consistência do banco restaurado.
- 7. Liberar o ambiente para utilização.

Esse procedimento busca minimizar o tempo de indisponibilidade e preservar o maior volume possível de informações.

Objetivos de Recuperação

Para orientar a estratégia de continuidade do serviço, são definidos os seguintes indicadores:

Indicador	Descrição
RPO (Recovery Point Objective)	Perda máxima aceitável de dados
RTO (Recovery Time Objective)	Tempo máximo aceitável para recuperação do ambiente

Como referência inicial para a plataforma, recomenda-se:

Indicador	Valor Inicial
RPO	Até 1 hora
RTO	Até 4 horas

Esses valores poderão ser revisados conforme os requisitos de disponibilidade da organização.

Teste de Recuperação

A realização periódica de teste de restauração é fundamental para garantir que os backups possam ser utilizados em situações reais.

Os testes deverão contemplar:

- Restauração completa do banco;
- Recuperação utilizando backups diferenciais;
- Aplicação dos logs de transações;
- Validação da integridade dos dados restaurados;
- Verificação do funcionamento das aplicações após a recuperação.

Os resultados dos testes deverão ser documentados e arquivados para fins de auditoria.

Segurança dos Backups

Os arquivos de backup contêm informações sensíveis e deverão receber proteção adequada.

Recomenda-se:

- Criptografia dos arquivos de backup;
- Controle de acesso baseado em perfis autorizados;
- Armazenamento em locais seguros;
- Monitoramento dos acessos aos arquivos;
- Registro das operações de criação, cópia e restauração.

Essas medidas reduzem o risco de acesso não autorizado e reforçam a conformidade com políticas de segurança da informação.

Monitoramento das Rotinas de Backup

As execuções de backup deverão ser monitoradas continuamente.

Entre as informações registradas destacam-se

- Data e hora da execução;
- Tipo de backup realizado;
- Tamanho do arquivo gerado;



- Tempo de execução;
- Usuário ou processo responsável;
- Resultado da operação (sucesso ou falha);
- Mensagens de erro, quando aplicável.

Esses registros permitem identificar falhas rapidamente e comprovar a execução das rotinas previstas.

Boas Práticas

Para garantir a efetividade da política de backup, recomenda-se:

- Automatizar as rotinas de backup;
- Validar periodicamente os arquivos gerados;
- Executar teste de restauração em ambiente de homologação;
- Manter cópias armazenadas em locais distintos;
- Revisar periodicamente a política de retenção dos backups;
- Documentar alterações nos procedimentos de recuperação;
- Monitorar a capacidade de armazenamento disponível.

Considerações Técnicas

A estratégia de backup e recuperação definida para a Audit Analytics Platform busca assegurar a continuidade dos serviços e a preservação das informações armazenadas no banco de dados.

A combinação de backups completos, diferenciais e logs de transações proporciona maior flexibilidade para recuperação do ambiente, reduzindo perdas de dados e o tempo de indisponibilidade.

Além disso, a realização periódica de teste de restauração e o monitoramento das rotinas de backup fortalecem a confiabilidade da solução e contribuem para a governança do ambiente de dados.



Versionamento dos Scripts

Objetivo

Este capítulo apresenta as diretrizes para o versionamento dos scripts utilizados na criação, manutenção e evolução do banco de dados da **Audit Analytics Platform**.

O versionamento permite controlar as alterações realizadas ao longo do ciclo de vida da solução, garantindo rastreabilidade, organização e segurança durante o desenvolvimento, homologação e implantação em produção.

A adoção de um processo estruturado de versionamento facilita a colaboração entre equipes, reduz riscos de inconsistências e possibilita a recuperação de versões anteriores quando necessário.

Objetivos do Versionamento

O processo de versionamento tem como principais objetivos:

- Registrar todas as alterações realizadas nos scripts;
- Garantir histórico completo das versões;
- Facilitar auditorias técnicas;
- Permitir reversão de alterações quando necessário;
- Controlar a evolução do banco de dados;
- Padronizar o processo de implantação;
- Apoiar a integração entre desenvolvimento, teste e produção.

Estrutura dos Scripts

Os scripts deverão se organizados em diretórios conforme sua finalidade.

Diretório	Conteúdo
01_Database	Criação do banco de dados
02_Tabelas	Scripts de criação das tabelas
03_Constraints	Chaves primárias, estrangeiras e restrições
04_Indices	Índices clustered e nonclustered



Diretório	Conteúdo
05_Views	Views analíticas
06_Procedures	Stored Procedures
07_Functions	Funções auxiliares (quando aplicável)
08_ETL	Scripts de carga e transformação
09_Testes	Scripts de validação
10_Deploy	Scripts completos de implantação

Essa organização facilita a localização dos arquivos e a execução das rotinas de implantação.

Convenção de Nomes

Todos os scripts deverão seguir um padrão único de nomenclatura.

Estrutura recomendada:

AAA_BBB_Descricao.sql

Onde:

- **AAA** representa a ordem de execução;
- **BBB** representa o tipo do objeto;
- **Descricao** identifica o conteúdo do script.

Exemplos

001_CreateDatabase.sql

010_Create_dCalendario.sql

020_Create_dProduto.sql

030_Create_fPerdas.sql

040_CreateIndexes.sql

050_CreateViews.sql

060_CreateProcedures.sql

070_ETL.sql



Essa padronização reduz erros durante a implantação e facilita a manutenção do projeto.

Controle de Versões

Cada alteração deverá resultar em uma nova versão dos scripts.

As versões deverão seguir um padrão de numeração progressiva.

Versão	Descrição
1.0	Primeira versão estável
1.1	Correções de bugs
1.2	Ajustes de desempenho
2.0	Inclusão de novos módulos
2.1	Melhorias nas procedures

Toda versão deverá ser acompanhada de documentação descrevendo as alterações realizadas.

Registro de Alterações (Change Log)

Cada nova versão deverá possuir um histórico contendo as modificações implementadas.

Exemplo:

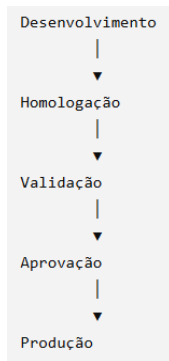
Versão	Data	Responsável	Alteração
1.0	10/07/2026	Equipe de Desenvolvimento	Criação inicial do banco
1.1	18/07/2026	Equipe de Desenvolvimento	Ajuste na tabela dProduto
1.2	24/07/2026	Equipe de Desenvolvimento	Inclusão de novos índices
2.0	30/07/2026	Equipe de Desenvolvimento	Inclusão das Stored Procedures

O histórico de alterações facilita auditorias e a identificação de mudanças ao longo do tempo.



Fluxo de Implantação

O processo de atualização do banco de dados deverá seguir um fluxo controlado.



Nenhuma alteração deverá ser aplicada diretamente no ambiente de produção sem validação prévia.

Repositório de Código

Todos os scripts deverão ser armazenados em um sistema de controle de versão.

Para este projeto, recomenda-se a utilização do **Git**, permitindo:

- Controle de alterações;
- Trabalho colaborativo;
- Recuperação de versões anteriores;
- Registro do histórico de desenvolvimento;
- Criação de ramificações (branches) para novas funcionalidades.

A estrutura sugerida do repositório é:



```
AuditAnalyticsPlatform/
|
|_ BancoDados/
|   |_ 01_Database/
|   |_ 02_Tabelas/
|   |_ 03_Constraints/
|   |_ 04_Indices/
|   |_ 05_Views/
|   |_ 06_Procedures/
|   |_ 07_Functions/
|   |_ 08_ETL/
|   |_ 09_Testes/
|   |_ 10_Deploy/
|
|_ Documentacao/
|   |_ NR-001.docx
|   |_ NR-002.docx
|   |_ NR-003.docx
|   |_ NR-004.docx
|   |_ NR-005.docx
|   |_ NR-006.docx
|
|_ README.md
```

Boas Práticas

Durante o versionamento dos scripts deverão ser observadas as seguintes recomendações:

- Manter scripts pequenos e organizados;
- Evitar alterações diretamente em produções;
- Registrar todas as mudanças realizadas;
- Utilizar comentários para explicar alterações relevantes;
- Revisar scripts antes da publicação;
- Executar testes em ambiente de homologação;
- Armazenar todas as versões no repositório oficial;
- Garantir que cada versão possa ser reproduzida.

Benefícios

A adoção de um processo estruturado de versionamento proporciona diversos benefícios:

- Maior organização do projeto;
- Rastreabilidade completa das alterações;
- Facilidade de manutenção;



- Redução do risco de erros durante implantações;
- Melhor colaboração entre equipes;
- Recuperação rápida de versões anteriores;
- Apoio às auditorias técnicas;
- Padronização das entregas.

Considerações Técnicas

O versionamento dos scripts constitui uma prática essencial para a governança do banco de dados da Audit Analytics Platform.

A utilização de convenções padronizadas, controle de versões e armazenamento em repositório Git permite acompanhar toda a evolução da solução, garantindo maior confiabilidade, segurança e facilidade de manutenção.

Além disso, a organização proposta facilita futuras expansões do projeto, contribuindo para sua escalabilidade e para a adoção de práticas modernas de desenvolvimento e administração de banco de dados.

Plano de Implantação

Objetivo

Este capítulo apresenta o plano de implantação do banco de dados da **Audit Analytics Platform**, estabelecendo as etapas necessárias para disponibilizar o ambiente em produção de forma controlada, segura e padronizada.

O plano tem como finalidade minimizar riscos durante a implantação, garantir a integridade das informações e assegurar que todos os componentes de banco de dados sejam instalados na sequência correta.

Além disso, define responsabilidades, pré-requisitos e critérios de validação para que o ambiente esteja apto a suportar as operações analíticas da plataforma.

Escopo da Implantação

O plano contempla a implantação dos seguintes componentes:

- Banco de dados da Audit Analytics Platform;
- Tabelas dimensionais;
- Tabela fato;



- Chaves primárias e estrangeiras;
- Constraints;
- Índices;
- Views;
- Stored Procedures;
- Scripts e ETL;
- Configurações de segurança;
- Permissões de acesso;
- Dados iniciais das dimensões.

Não fazem parte deste plano a instalação do Microsoft SQL Server, do Power BI ou de sistemas externos que servirão como origem dos dados.

Pré-requisitos

Antes da implantação deverão ser verificados os seguintes requisitos:

Infraestrutura

- Microsoft SQL Server instalado e configurado;
- Espaço em disco suficiente para o banco de dados;
- Memória e processamento compatíveis com a carga esperadas;
- Rede operacional entre os sistemas envolvidos.

Software

- SQL Server Management Studio (SSMS) ;
- Ferramentas de ETL (quando aplicável);
- Ambiente de desenvolvimento homologado.

Segurança

- Usuário administrador do banco de dados;
- Credenciais de acesso válidas;
- Política de backup configurada;
- Permissões previamente definidas,



Ambientes

A implantação deverá respeitar a separação entre ambientes.

Ambiente	Finalidade
Desenvolvimento	Criação e evolução dos scripts
Homologação	Validação funcional e técnica
Produção	Operação oficial da plataforma

Nenhuma alteração deverá ser aplicada diretamente no ambiente de produção sem testes prévios em homologação.

Sequência de Implantação

A implantação deverá seguir rigorosamente a ordem apresentada abaixo.

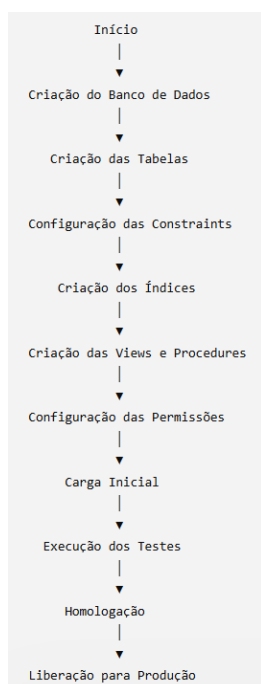
Ordem	Etapas
1	Criação do banco de dados
2	Configuração dos arquivos de dados e log
3	Criação das tabelas dimensionais
4	Criação da tabela fato
5	Criação das chaves primárias
6	Criação das chaves estrangeiras
7	Aplicação das constraints
8	Criação dos índices
9	Criação das views
10	Criação das Stored Procedures
11	Configuração das permissões
12	Carga inicial das dimensões



Ordem	Etapa
13	Carga inicial da tabela fato
14	Execução dos testes de validação
15	Liberação para utilização

Essa sequência garante que as dependências entre os objetos sejam respeitadas.

Fluxo da Implantação



Carga Inicial

Após a criação da estrutura física do banco de dados, deverá ser realizada a carga inicial das tabelas.

A ordem recomendada é:

1. dCalendario
2. dEstado
3. sSupervisor
4. dGerente
5. dCategoria



6. dLoja
7. dProduto
8. fPerdas

A carga da tabela fato somente deverá ocorrer após a conclusão da carga de todas as dimensões.

Validação da Implantação

Ao término da implantação, deverão ser executadas verificações para confirmar que todos os componentes foram criados corretamente.

Entre as validações recomendadas estão:

- Existência de todas as tabelas;
- Existência de todas as chaves primárias;
- Existência das chaves estrangeiras;
- Funcionamento das views;
- Execução das Stored Procedures;
- Validação dos índices;
- Integridade referencial;
- Quantidade de registros carregados;
- Teste de conexão do Power BI.

Somente após a aprovação dessas verificações o ambiente poderá ser considerado apto para uso.

Plano de Contingência

Caso seja identificada alguma falha durante a implantação, deverão ser adotadas as seguintes ações:

1. Interromper imediatamente o processo.
2. Registrar a ocorrência.
3. Restaurar o backup realizado antes da implantação.
4. Corrigir os scripts responsáveis pela falha.
5. Executar novamente os teste em homologação.



6. Reagendar a implantação.

Esse procedimento reduz o risco de indisponibilidade e preserva a integridade do ambiente.

Responsabilidades

Responsável	Atividade
Administrador de Banco de Dados (DBA)	Implantação da estrutura do banco de dados
Desenvolvedor	Disponibilização e validação dos scripts
Equipe de Infraestrutura	Disponibilização do ambiente
Analista de Dados	Validação das cargas e consultas
Gestor do Projeto	Aprovação da implantação

A definição clara das responsabilidades contribui para uma implantação organizada e reduz a possibilidade de falhar operacionais.

Critérios de Sucesso

A implantação será considerada concluída com sucesso quando forem atendidos os seguintes critérios:

- Todos os objetos do banco de dados criados corretamente;
- Carga inicial executada sem erros;
- Integridade referencial validada;
- Procedures executadas com sucesso;
- Views retornando resultado esperados;
- Índices criados conforme especificação;
- Power BI conectado ao banco de dados;
- Teste funcionais aprovados;
- Backup inicial realizado;
- Aprovação formal da equipe responsável.



Cronograma Sugerido

Etapa	Duração Estimada
Preparação do ambiente	1 hora
Implantação da estrutura	1 hora
Carga inicial	2 horas
Validações	1 hora
Homologação	2 horas
Liberação para produção	30 minutos

O cronograma poderá ser ajustado de acordo com a infraestrutura disponível e o volume inicial de dados.

Boas Práticas

Durante a implantação deverão ser observadas as seguintes recomendações:

- Executar backup completo antes de qualquer alteração;
- Validar previamente todos os scripts;
- Implantar em janela de manutenção;
- Monitorar logs durante a execução;
- Registrar todas as ocorrências;
- Confirmar a integridade dos dados após cada etapa;
- Manter um plano de reversão atualizado;
- Documentar todas as alterações realizadas.

Considerações Técnicas

O plano de implantação da Audit Analytics Platform foi elaborado para garantir uma transição segura entre os ambientes de desenvolvimento, homologação e produção.



A sequência estruturada de atividades, aliada às validações, ao controle de risco e às responsabilidades definidas, contribui para uma implantação confiável e alinhada às boas práticas de administração de banco de dados.

A adoção deste plano reduz a probabilidade de falhas operacionais, facilita a rastreabilidade das atividades executadas e estabelece uma base sólida para futuras evoluções da plataforma.

Testes e Critérios de Validação

Objetivo

Este capítulo apresenta a estratégia de teste e os critérios de validação adotados para o banco de dados da **Audit Analytics Platform**.

O objetivo é assegurar que todos os componentes da solução funcionem conforme os requisitos definidos, garantindo a integridade dos dados, o correto funcionamento das rotinas de carga, o desempenho das consultas e a confiabilidade das informações disponibilizadas aos usuários.

A execução sistemática dos testes reduz o risco de falhas em produção e contribui para a qualidade geral da plataforma.

Estratégia de Teste

Os testes deverão ser realizados em ambiente de homologação, utilizando uma base de dados representativa do ambiente de produção.

A estratégia contempla a validação dos seguintes componentes:

- Estrutura do banco de dados;
- Tabelas dimensionais;
- Tabela fato;
- Relacionamentos;
- Constraints;
- Índices;
- Views;
- Stored Procedures;
- Processos ETL;



- Integração com a Microsoft Power BI.

Cada etapa deverá se documentada e registrada para fins de auditoria.

Tipos de Teste

Os testes estruturais verificam a criação correta dos objetos do banco de dados.

Itens avaliados:

- Existência das tabelas;
- Estrutura das colunas;
- Tipos de dados;
- Chave primárias;
- Chaves estrangeiras;
- Constraints;
- Índices;
- Views;
- Stored Procedures.

Teste Funcionais

Os testes funcionais verificam se as operações executam conforme esperado.

Exemplos:

- Inserção de registros;
- Atualização de dados;
- Exclusão controlada;
- Execução das procedures;
- Funcionamento das views;
- Consultas analíticas.

Teste de Integridade

Os testes de integridade garantem que as regras de negócio sejam respeitadas.

Entre os principais teste estão:



- Validação das chaves estrangeiras;
- Verificação de registros órfãos;
- Respeito às constraints;
- Validação de campos obrigatórios;
- Verificação de duplicidades;
- Consistência entre dimensões e tabela fato.

Testes de Desempenho

Os testes de desempenho avaliam a eficiência do banco de dados sob diferentes cargas de trabalho.

Serão analisados:

- Tempo de resposta das consultas;
- Execução das views;
- Performance das procedures;
- Tempo de carga do ETL;
- Atualização dos dashboards do Power BI.

Teste de Recuperação

Os testes de recuperação verificam a eficácia dos procedimentos de backup e restauração.

Serão avaliados:

- Restauração do backup completo;
- Aplicação do backup diferencial;
- Recuperação por log de transações;
- Validação da integridade após restauração.

Casos de Teste

A tabela a seguir apresenta exemplos dos principais casos de testes previstos.



Código Caso de Teste	Resultado Esperado
CT-001 Criar banco de dados	Banco criado com sucesso
CT-002 Criar tabelas	Todas as tabelas criadas
CT-003 Criar índices	Índices disponíveis
CT-004 Executar procedure de carga	Dados carregados corretamente
CT-005 Consultar view analítica	Dados retornados corretamente
CT-006 Inserir registro inválido	Operação rejeitada
CT-007 Executar ETL	Processo concluído sem erros
CT-008 Atualizar Power BI	Dados atualizados
CT-009 Restaurar backup	Banco recuperado corretamente
CT-010 Validar integridade referencial	Nenhuma inconsistência encontrada

Critérios de Aprovação

Um teste será considerado aprovado quando:

- O resultado obtido corresponder ao resultado esperado;
- Não ocorreram erros críticos durante a execução;
- Os dados permaneceram íntegros após a operação;
- As regras de negócio forem respeitadas;
- O desempenho permanecer dentro dos limites estabelecidos.

Caso qualquer critério não seja atendido, o teste deverá ser classificado como reprovado e reexecutado após a correção da falha.

Registro dos Resultados

Todos os testes executados deverão ser registrados em documentação específica.

Cada registro deverá conter:

- Código de teste;



- Objetivo;
- Responsável pela execução;
- Data da execução;
- Ambiente utilizado;
- Resultado esperado;
- Resultado obtido;
- Evidências (capturas de tela, logs ou consultas SQL);
- Status (Aprovado ou Reprovado);
- Observações.

Essa documentação garante rastreabilidade e facilita futuras auditorias.

Tratamento de Não Conformidades

Quando um teste apresentar resultados diferentes do esperado, deverão ser seguidas as etapas abaixo:

1. Registrar a ocorrência.
2. Identificar a causa da falha.
3. Corrigir o problema.
4. Atualizar os scripts, quando necessário.
5. Reexecutar o teste.
6. Registrar o novo resultado.

Somente após a aprovação o item poderá ser considerado concluído.

Indicadores de Qualidade

Durante a fase de validação recomenda-se acompanhar os seguintes indicadores:

Indicador	Objetivo
Percentual de testes aprovados	Avaliar estabilidade da solução
Número de falhas encontradas	Medir qualidade da implantação
Tempo médio de execução dos testes	Avaliar eficiência da validação



Indicador	Objetivo
Quantidade de retrabalhos	Identificar necessidade de melhorias
Tempo de resposta das consultas	Avaliar desempenho do banco de dados

Esses indicadores auxiliam na tomada de decisão quanto à liberação da solução para produção.

Critérios para Liberação em Produção

O ambiente será considerado apto para produção quando:

- Todos os testes estruturais forem aprovados;
- Todos os testes funcionais forem aprovados;
- Não existirem falhas críticas em aberto;
- Os processos ETL executam corretamente;
- A integridade referencial estiver preservada;
- O Power BI consumir os dados sem inconsistências;
- Os backups estiverem configurados e validados;
- A documentação estiver concluída e aprovada.

Boas Práticas

Para garantir a qualidade do processo de validação recomenda-se:

- Executar os testes em ambiente de homologação;
- Utilizar massa de dados representativa;
- Automatizar testes repetitivos sempre que possível;
- Documentar todos os resultados;
- Revisar periodicamente os casos de teste;
- Reexecutar testes após alterações estruturais;
- Manter histórico das validações realizadas.



Considerações Técnicas

A estratégia de testes definida para a Audit Analytics Platform busca assegurar que todos os componentes do banco de dados atendam aos requisitos funcionais e não funcionais especificados ao longo do projeto.

A execução sistemática dos testes estruturais, funcionais, de desempenho, integridade e recuperação contribui para a entrega de um ambiente confiável, seguro e preparado para suportar as operações analíticas da plataforma.

Os critérios de validação apresentados neste capítulo estabelecem um processo padronizado para aprovação das entregas e reduzem significativamente os riscos associados à implantação em produção.

Considerações Finais

Encerramento do Projeto

O desenvolvimento do banco de dados da **Audit Analytics Platform** representa a consolidação de uma arquitetura estruturada para armazenamento, organização e análise de informações relacionadas aos processos de auditoria de perdas.

Ao longo deste documento foram apresentadas as etapas de modelagem, implementação, governança e administração do banco de dados, contemplando desde a definição da estrutura física até os procedimentos necessários para sua implantação, manutenção e evolução.

A solução foi projetada com base em um modelo dimensional, utilizando tabelas de dimensão e uma tabela fato para proporcionar maior desempenho em consultas analíticas, simplificar a construção de indicadores e facilitar a integração com ferramentas de Business Intelligence.

Objetivos Alcançados

Os principais objetivos estabelecidos para o banco de dados foram alcançados por meio de implementação de uma estrutura organizada, escalável e aderente às boas práticas de modelagem de dados.

Entre os resultados obtidos destacam-se:

- Desenvolvimento de um modelo dimensional voltado para análise de auditoria;
- Organização das informações em dimensões e tabela fato;
- Definição de chaves primárias, estrangeiras e restrições de integridade;



- Implementação de índices para otimização das consultas
- Criação de views analíticas para simplificar o acesso aos dados;
- Definição de Stored Procedures para automatização de processos;
- Estruturação de estratégias de ETL;
- Definição de políticas de monitoramento, manutenção e backup;
- Planejamento do processo de implantação e validação da solução;
- Padronização da documentação técnica do banco de dados.

Esses elementos estabelecem uma base consistente para utilização da plataforma em ambientes corporativos.

Benefícios da Solução

A arquitetura proposta oferece benefícios técnicos e operacionais que contribuem para a eficiência dos processos de auditoria.

Entre os principais benefícios estão:

- Centralização das informações em um ambiente único;
- Redução da redundância de dados;
- Melhoria da qualidade das informações utilizadas nas análises;
- Agilidade na geração de relatórios gerenciais;
- Suporte à criação de dashboards no Microsoft Power BI;
- Facilidade para identificação de padrões e tendências de perdas;
- Escalabilidade para crescimento do volume de dados;
- Maio rastreabilidade dos processos de carga e atualização;
- Estrutura preparada para futuras integrações com novos sistemas.

Contribuição para a Gestão

A utilização da Audit Analytics Platform possibilita que gestores tenham acesso a informações consolidadas e atualizadas para apoiar a tomada de decisão.

Com a organização dos dados em um modelo dimensional, torna-se possível:

- Acompanhar indicadores de perdas por período;



- Analisar o desempenho por loja;
- Identificar produtos com maior incidência de perdas;
- Avaliar resultados por categoria;
- Acompanhar indicadores por gerente e supervisor;
- Comparar períodos distintos;
- Apoiar ações de melhoria contínua.

Essas funcionalidades fortalecem a utilização da análise de dados como instrumento de apoio à gestão.

Escalabilidade e Evolução

O modelo desenvolvido foi concebido para permitir futuras expansões sem necessidade de alteração estruturais significativas.

Entre as possibilidades de evolução destacam-se:

- Inclusão de novas dimensões analíticas;
- Integração completa dos processos ETL;
- Atualização em tempo quase real dos dados;
- Implementação de indicadores avançados;
- Adoção de modelos preditivos utilizando técnicas de Inteligência Artificial e Machine Learning;
- Integração com serviços em nuvem para aumento da disponibilidade e escalabilidade.

Essa flexibilidade amplia o potencial de utilização da plataforma em diferentes cenários organizacionais.

Governança de Dados

A documentação elaborada para o banco de dados contribui para a adoção de práticas de governança, estabelecendo padrões para criação, manutenção e evolução da estrutura de dados.

A definição de processo de versionamento, controle de alterações, monitoramento e validação fortalece a confiabilidade do ambiente e facilita sua administração ao longo do tempo.



Além disso, a padronização adotada favorece a integração entre equipes de desenvolvimento, administração de banco de dados e análise de negócios.

Limitações

Embora a arquitetura proposta tenha sido desenvolvida para atender aos requisitos definidos para a Audit Analytics Platform, algumas funcionalidades poderão ser incorporadas em versões futuras.

Entre elas:

- Implementação de cargas em tempo real;
- Replicação entre ambientes;
- Particionamento de grandes tabelas;
- Auditoria detalhada de alterações (*Change Data Capture*);
- Criptografia de dados sensíveis em nível de coluna;
- Integração com ferramentas avançadas de monitoramento.

Essas funcionalidades poderão ser avaliadas conforme a evolução da plataforma e as necessidades da organização.

Conclusão

O banco de dados desenvolvido para a **Audit Analytics Platform** estabelece uma infraestrutura sólida para o armazenamento e a análise de informações relacionadas aos processos de auditoria de perdas.

A utilização de um modelo dimensional, associado às estratégias de integridade, desempenho, segurança e governança apresentadas neste documento, proporciona uma base consistente para o desenvolvimento de soluções analíticas voltadas ao apoio à decisão.

Espera-se que a estrutura proposta contribua para a melhoria contínua dos processos de auditoria, permitindo maior eficiência operacional, qualidade das informações e capacidade analítica, além de servir como referência para futuras evoluções da plataforma.

Referências Bibliográficas

As referências apresentadas a seguir contemplam normas, livros e documentações técnicas utilizadas como base par ao desenvolvimento do banco de dados da **Audit Analytics Platform**, abrangendo conceitos de modelagem



dimensional, administração de bancos de dados, linguagem SQL, Business Intelligence e governança de dados.

Livros

KIMBALL, Ralph; ROSS, Margy. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. ed. Indianapolis: John Wiley & Sons, 2013.

DATE, C. J. *An Introduction to Database Systems*. 8. ed. Boston: Addison-Wesley, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. *Fundamentals of Database Systems*. 7. ed. Boston: Pearson, 2017.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Database System Concepts*. 7. ed. New York: McGraw-Hill, 2020.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Software Engineering: A Practitioner's Approach*. 9. ed. New York: McGraw-Hill, 2019.

SOMMERVILLE, Ian. *Software Engineering*. 10. ed. Boston: Pearson, 2016.

Documentação Oficial

Microsoft. SQL Server Documentation. Disponível em:
<https://learn.microsoft.com/sql/>

Acesso em: 30 jul. 2026.

Microsoft. SQL Server Transact-SQL Reference. Disponível em:
<https://learn.microsoft.com/sql/t-sql/>

Acesso em: 30 jul. 2026.

Microsoft. SQL Server Integration Services (SSIS). Disponível em:
<https://learn.microsoft.com/sql/integration-services/>

Acesso em: 30 jul. 2026.

Microsoft. Power BI Documentation. Disponível em:
<https://learn.microsoft.com/power-bi/>

Acesso em: 30 jul. 2026.



Microsoft. SQL Server Best Practices. Disponível em:
<https://learn.microsoft.com/sql/sql-server/>

Acesso em: 30 jul. 2026.

Governança e Gerenciamento de Dados

DAMA INTERNATIONAL.

The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK2).

2. ed.

Technics Publications, 2017.

ISO/IEC 25010:2011.

Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE).

International Organization for Standardization.

Controle de Versionamento

CHACON, Scott; STRAUB, Ben.

Pro Git.

ed.

Apress, 2014.

Disponível em:

<https://git-scm.com/book/>

Engenharia de Dados e Business Intelligence

KIMBALL GROUP.

The Kimball Group Reader.

Indianapolis: Wiley, 2015.

MICROSOFT.

Power BI Guidance Documentation.



Disponível em:

<https://learn.microsoft.com/power-bi/guidance/>

Normas Técnicas

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS.

ABNT NBR 6023:2018 — Informação e documentação — Referências —
Elaboração.

Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS.

ABNT NBR 14724:2024 — Informação e documentação — Trabalhos acadêmicos
— Apresentação.

Rio de Janeiro: ABNT.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS.

ABNT NBR ISO/IEC 25010 — Engenharia de software — Modelo de qualidade.

Considerações sobre as Referências

As obras e documentações apresentada fundamentam os conceitos de modelagem dimensional, administração de banco de dados relacionais, Business Intelligence, governança de dados e engenharia de software empregados no desenvolvimento da Audit Analytics Platform.

A consulta a essas fontes permitiu alinhar a estrutura do banco de dados às boas práticas adotadas pela indústria, contribuindo para a criação de uma solução consistente, escalável e preparada para futuras evoluções.

